

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕНА К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ ПЛАНИРОВАНИЯ ЗАДАЧ С
УЧЁТОМ ИХ МЕСТОПОЛОЖЕНИЯ И ВРЕМЕННЫХ ПАРАМЕТРОВ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная
инженерия»

Студент: _____ / Тютичкин Семен Владимирович, 221–329/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Васильев Денис Борисович/
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Системная и программная инженерия»

Тема ВКР	Разработка веб-приложения для планирования задач с учётом их местоположения и временных параметров
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	Автоматизированное формирование оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.
Основные функции	1. Управление задачами: создать, изменить, удалить 2. Оптимизация плана выполнения задач 3. Web-интерфейс
Используемые технологии и платформы	Golang, JavaScript, TypeScript, React, Docker, PostgreSQL, HTML, CSS
ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Исследовать проблему оптимизации выполнения задач с гео-параметрами и временными параметрами 2. Проанализировать существующие продукты для оптимизации выполнения задач с гео-параметрами и временными параметрами 3. Разработать свое приложение на основе анализа существующих продуктов

ПЛАН РАБОТЫ НАД ВКР

Этапы	Недели семестра																	
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
Анализ предметной области																		
Выбор технологий для решения проблем предметной области																		
Проектирование системы																		
Разработка веб-приложения																		
Тестирование веб-приложения																		
Конфигурация развертывания системы																		

РУКОВОДИТЕЛЬ ОП:

«__»____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«__»____2026, _____ / Васильев Денис Борисович /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«__»____2026, _____ / Тютичкин Семен Владимирович, 221–329/ /
подпись *ФИО, группа*

АННОТАЦИЯ

Наименование работы: Разработка веб-приложения для автоматизированного планирования маршрутов с учетом временных окон.

Цель работы: разработка веб-приложения для автоматизированного формирования оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.

Задачи работы:

1. проанализировать предметную область и существующие программные решения;
2. исследовать алгоритмы маршрутизации с временными окнами;
3. разработать архитектуру программной системы;
4. реализовать серверную часть и алгоритм оптимизации маршрута;
5. разработать пользовательский интерфейс для планирования задач, визуализации маршрута, импорта и экспорта задач;
6. реализовать интеграцию с внешними картографическими сервисами;
7. провести тестирование и оценить производительность.

Объект исследования – процесс планирования последовательности выполнения задач в городской среде с учетом пространственных и временных параметров.

Предмет исследования – алгоритмы маршрутизации и методы оптимизации порядка выполнения задач с временными окнами.

Объем работы составляет 79 страниц, в том числе 18 страниц приложений. Работа содержит введение, 3 главы, заключение и 3 приложения; включает 25 рисунков, 13 таблиц и 11 листингов. В приложения вынесены 10 листингов; в основной части размещен 1 листинг с псевдокодом алгоритма. Библиографический список включает 61 источник, из которых 30 печатных и 31 интернет-источник.

В первой главе выполнены анализ предметной области, постановка задачи, обоснование выбора технологий и сравнительный анализ аналогов. Во второй главе представлены модель данных, архитектура серверной части, алгоритм оптимизации маршрута, проект пользовательского интерфейса и техническая реализация. В третьей главе раскрыты вопросы внедрения и опытной эксплуатации, аспекты отказоустойчивости, тестирование и результаты нагрузочного анализа производительности. В заключении приведены выводы, оценка результатов, практическая значимость и перспективы развития системы.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ	11
1.1 Исходное состояние объекта и характеристика предметной области	11
1.2 Постановка проблемы и формирование цели разработки	12
1.3 Обоснование выбора технологий и платформ	14
1.4 Анализ алгоритмических подходов к задаче маршрутизации с временными окнами	17
1.5 Обзор аналогов и сравнительный анализ	20
1.6 Выводы	21
2 РЕАЛИЗАЦИЯ	23
2.1 Модель данных	23
2.2 Архитектура системы	26
2.3 Алгоритм оптимизации маршрута	28
2.4 Интерфейс	35
2.5 Техническая реализация	43
2.6 Выводы	45
3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ	47
3.1 Анализ внедрения продукта	47
3.2 Аспекты отказоустойчивости, надежности и производительности	48
3.3 Тестирование	59
3.4 Внедрение и сопровождение программного продукта	61
3.5 Ограничения	61
3.6 Выводы	61
ЗАКЛЮЧЕНИЕ	63
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	66
ПРИЛОЖЕНИЕ А	72

ПРИЛОЖЕНИЕ Б	76
ПРИЛОЖЕНИЕ В	86

ВВЕДЕНИЕ

Актуальность темы обусловлена ростом числа повседневных и профессиональных задач, привязанных одновременно к географическому адресу и временному окну. Ручное планирование таких маршрутов трудоемко и подвержено ошибкам, что формирует устойчивую потребность в инструментах автоматизированного планирования личной и профессиональной деятельности [2]. Та же тенденция проявляется в коммерческой логистике последней мили [1]: рост числа адресов доставки и сужение временных окон обслуживания делают построение оптимального маршрута экономически значимым.

Существующие цифровые решения представлены либо приложениями для управления задачами, ориентированными на учет перечня дел без учета пространственного фактора, либо навигационными сервисами, формирующими маршрут без учета длительности выполнения задач. В результате пользователи вынуждены комбинировать несколько приложений, вручную сопоставляя адреса, интервалы времени и порядок посещения точек. Подобный подход характеризуется высокой трудоемкостью и ростом вероятности ошибок при увеличении количества задач [3]. Специализированные логистические платформы, напротив, ориентированы на крупный коммерческий сегмент и недоступны частному пользователю без коммерческой подписки.

Объект исследования – процесс планирования последовательности выполнения задач в городской среде с учетом пространственных и временных параметров.

Предмет исследования – алгоритмы маршрутизации и методы оптимизации порядка выполнения задач с временными окнами в условиях, характерных для частного пользователя и малого бизнеса.

Цель работы – разработка веб-приложения для автоматизированного формирования оптимальной последовательности выполнения задач с учетом географического положения, временных окон и длительности выполнения.

Для достижения поставленной цели сформулированы следующие задачи:

1. проанализировать предметную область и существующие программные решения, выявить ограничения существующих инструментов;
2. исследовать алгоритмические подходы к маршрутизации с временными окнами, обосновать выбор алгоритма;
3. разработать архитектуру программной системы и модель данных;
4. реализовать серверную часть и алгоритм оптимизации маршрута с поддержкой дополнительных ограничений;
5. разработать пользовательский интерфейс для планирования задач, визуализации маршрута на карте, импорта и экспорта задач;
6. реализовать интеграцию с внешними картографическими сервисами;
7. провести модульное и нагрузочное тестирование разработанного решения;
8. оценить производительность и масштабируемость системы.

Научная и практическая новизна работы состоит в объединении в одном инструменте управления задачами, автоматической оптимизации маршрута с учетом временных окон и попарных ограничений порядка выполнения и визуализации маршрута. Такое сочетание функций отсутствует у рассмотренных бесплатных решений для индивидуального пользователя.

Практическая значимость работы состоит в том, что перечисленные возможности становятся доступными частным пользователям, специалистам на выезде, курьерам малого бизнеса и индивидуальным предпринимателям без коммерческой подписки. Веб-форма распространения и контейнеризация на основе Docker Compose обеспечивают воспроизводимое развертывание на типовом серверном оборудовании.

Методы исследования: анализ предметной области и сравнительный анализ существующих решений; обзор и сравнение алгоритмических под-

ходов к задаче маршрутизации с временными окнами. При проектировании применены методы объектно-ориентированного проектирования — принципы чистой архитектуры [4], предметно-ориентированное проектирование [5] и паттерны проектирования [6]. Качество решения подтверждено модульным и нагрузочным тестированием [7–8] и экспериментальной оценкой производительности.

Структура работы. Работа состоит из введения, трех глав, заключения, списка использованных источников и трех приложений. В первой главе выполнены анализ предметной области и обоснование выбора технологий. Во второй главе представлены проектные решения и реализация. В третьей главе рассмотрены вопросы внедрения, отказоустойчивости и тестирования. В приложениях приведены листинги SQL-миграций и конфигурации, листинги ключевых фрагментов программного кода, а также техническое задание на разработку.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

1.1 Исходное состояние объекта и характеристика предметной области

При наличии у задач временных окон (например, «с 9:00 до 12:00») пользователь вынужден вручную сопоставлять время перемещения, длительность выполнения и интервалы доступности каждой точки [9–10], а с ростом числа задач трудоемкость такого планирования и вероятность ошибок быстро увеличиваются [3].

Целевой аудиторией системы являются:

1. частные пользователи, планирующие личные дела в городе;
2. мастера и специалисты на выезде;
3. курьеры малого бизнеса;
4. индивидуальные предприниматели;
5. диспетчеры, формирующие небольшие маршруты вручную.

Перечисленные категории различаются по составу задач и условиям работы, что определяет востребованность конкретных функций системы. В таблице 1 сопоставлены типичные сценарии использования и наиболее значимые для них функции по ключевым категориям пользователей.

Таблица 1 – Сопоставление категорий пользователей, сценариев и востребованных функций

Категория	Типичный сценарий	Наиболее востребованные функции
Частный пользователь	Личные дела за день: банк, МФЦ, врач, магазины с ограниченными часами работы	Временные окна, оптимизация порядка, выбор способа передвижения, карта
Мастер на выезде	Заявки с интервалами вида «к клиенту с 10 до 12»	Временные окна, длительность обслуживания, фиксация старта и финиша, оптимизация
Курьер малого бизнеса	Забор и доставка посылок с учетом окон получателей	Старт со склада и финиш, ограничения «А до В», импорт списка адресов, длительность обслуживания
Торговый представитель	Регулярные однотипные объезды фиксированного набора точек	Импорт и экспорт задач, клонирование задач, детерминированность результата

Особенностью данной предметной области является необходимость одновременного учета:

1. географического расположения объектов;
2. временных ограничений;
3. длительности выполнения задач;
4. минимизации времени перемещения и ожидания.

1.2 Постановка проблемы и формирование цели разработки

Анализ исходного состояния показал, что основной проблемой является отсутствие доступного инструмента, объединяющего управление задачами и автоматическую маршрутизацию с учетом временных окон.

Ручной подход:

1. требует значительных временных затрат;
2. не гарантирует оптимальность маршрута;
3. становится неэффективным при увеличении числа точек;
4. не обеспечивает формализованного учета временных ограничений.

Целью разработки является создание веб-приложения, обеспечивающего автоматизированное формирование оптимального маршрута выполнения задач с учетом их географического положения, временных окон и длительности выполнения.

На основании цели и выявленных потребностей целевой аудитории (таблица 1) сформирован следующий перечень функций:

1. создание, редактирование, удаление и клонирование задач;
2. импорт задач из файлов формата CSV и XLSX (Excel);
3. экспорт задач в форматах CSV, XLSX (Excel и JSON);
4. получение матрицы расстояний через внешние API;
5. вычисление оптимальной последовательности выполнения задач;
6. поддержка дополнительных ограничений маршрута: фиксация начальной и конечной точки, задание порядка выполнения пар задач;
7. выбор способа передвижения (автомобиль, пешком, общественный транспорт);
8. расчет времени прибытия, ожидания и завершения для каждой остановки;
9. визуальное обнаружение конфликтов временных окон;
10. визуализация маршрута на карте.

Обоснование отдельных функций. Ряд функций обусловлен особенностями работы целевой аудитории. Фиксация начальной и конечной точки отражает то, что реальный маршрут редко начинается и завершается в произвольном месте: курьер выезжает со склада и возвращается домой, мастер — из мастерской, поэтому точки старта и финиша задаются явно. Ограничения порядка «задача А до задачи В» выражают логические зависимости между делами (забрать посылку до доставки, посетить банк до оплаты поставщику), без учета которых минимальный по времени порядок может оказаться практически неисполнимым. Выбор способа передвижения учитывает разнородность аудитории по мобильности: частный пользователь в центре города

перемещается пешком или общественным транспортом, тогда как курьер использует автомобиль, а от выбранного режима зависят матрица расстояний и время в пути. Импорт и экспорт задач в форматах CSV и XLSX ориентированы на курьеров и торговых представителей, получающих адреса списком, для которых ручной ввод десятков точек нерационален.

Для обеспечения соответствия назначению определены следующие нефункциональные требования:

1. время ответа сервера — не более 2 секунд при типовой нагрузке 5 запросов в секунду;
2. поддержка современных браузеров (Google Chrome версии 90 и выше, Яндекс Браузер версии 25 и выше);
3. масштабируемость до 100 одновременных пользователей;
4. устойчивость к повторным запросам внешних API за счет кэширования.

1.3 Обоснование выбора технологий и платформ

На основании сформированных функциональных и нефункциональных требований обоснован выбор серверной и клиентской платформ, системы управления базами данных и внешнего картографического сервиса.

Выбор серверной платформы. В качестве языка для серверной части выбран Go [11] со следующими характеристиками, существенными для разрабатываемого приложения:

1. статическая типизация, снижающая риск ошибок на этапе компиляции;
2. встроенная модель конкурентности на основе горутин [12], обеспечивающая обслуживание большого числа одновременных HTTP-соединений без выделения отдельного потока операционной системы на каждый запрос;

3. компиляция в нативный исполняемый файл, что дает малый объем образа контейнера и предсказуемое потребление ресурсов.

Выбор системы управления базами данных. В качестве СУБД выбрана PostgreSQL [13]. Обоснование:

1. типизированные временные типы (TIMESTAMPTZ, DATE, TIME), удобные для хранения временных окон задач;
2. непрерывное промышленное применение с 1996 года, стабильный релизный цикл и соответствие стандарту SQL:2016 [13];
3. открытая лицензия и отсутствие лицензионных платежей при коммерческой эксплуатации.

Выбор клиентских технологий. Для реализации пользовательского интерфейса выбран язык TypeScript [14] с библиотекой React [15]. Причины выбора:

1. компонентный подход, позволяющий повторно использовать элементы интерфейса (карточка задачи, форма, диалог);
2. развитая экосистема готовых компонентов и картографических библиотек;
3. статическая типизация TypeScript, повышающая качество и понятность кода по сравнению с JavaScript [16].

Выбор внешнего картографического сервиса. Для геокодирования адресов и визуализации маршрута выбран сервис «Яндекс JavaScript API и HTTP Геокодер» [17] (далее — Яндекс JS API). В таблице 2 представлено сравнение доступных картографических решений.

Таблица 2 – Сравнение картографических API

Критерий	Яндекс JS API	Google Maps JS API	OpenStreetMap
Геокодер на русском языке	Да, включая топонимику РФ	Да	Качество в РФ ниже коммерческих провайдеров; обновление зависит от сообщества
Бесплатный лимит (геокодер)	1 000 за-пр./сутки	200 \$ кредит/мес.	Без ограничений
Работа в России	Без ограничений	Может быть недоступен	Без ограничений

На основании сравнения (таблица 2) для целевой аудитории (российские пользователи) выбран Яндекс JS API: это единственный из рассмотренных вариантов, который одновременно поддерживает российскую топонимику, предоставляет встроенный мультимаршрутизатор для построения матрицы расстояний и гарантированно доступен в российском сетевом сегменте.

Итоговый технологический стек. Принятые решения по всем компонентам сведены в таблицу 3.

Таблица 3 – Итоговый технологический стек разрабатываемой системы

Уровень	Технология	Обоснование
1	2	3
Серверная часть	Go 1.23 [18]	Компилируемый язык, простая модель конкурентности
База данных	PostgreSQL 16 [13]	Реляционность, поддержка типизированных временных типов, зрелость платформы
Клиентская часть	React + TypeScript [14]	Компонентный подход, статическая типизация, развитая экосистема
Сборка клиентской части	Vite 6 [19]	Оптимизированная сборка для производственной среды

Продолжение таблицы 3

1	2	3
Картографический API	Яндекс JS API 2.1	Русскоязычный гео-кодер, встроенный мультимаршрутизатор
Контейнеризация	Docker Compose [20]	Воспроизводимость среды, автоматическое применение миграций

1.4 Анализ алгоритмических подходов к задаче маршрутизации с временными окнами

Решаемая задача относится к классу «задача коммивояжера с временными окнами» (Travelling Salesman Problem with Time Windows, TSPTW) — обобщению классической задачи коммивояжера [21], в котором каждой вершине маршрута сопоставлен допустимый интервал начала обслуживания. Пользователь задает перечень адресов с временными окнами и длительностью обслуживания, а система должна построить порядок посещения, минимизирующий суммарное время маршрута с учетом ожиданий перед открытием окон. Прибытие в вершину раньше открытия окна влечет ожидание; прибытие после закрытия окна делает обслуживание невозможным. Поскольку при отсутствии временных ограничений TSPTW сводится к классической задаче коммивояжера, она наследует его NP-трудность [22–25] и в общем случае не имеет известного полиномиального точного алгоритма решения.

Формализуем задачу. Пусть задан граф $G = (V, E)$, где $V = \{v_0, v_1, \dots, v_n\}$ — множество узлов (задач пользователя), E — множество дуг с матрицей расстояний d_{ij} и матрицей времен перемещения t_{ij} .

Каждый узел v_i ($i \geq 1$) характеризуется:

1. временным окном $[a_i, b_i]$ — допустимым интервалом начала обслуживания;
2. длительностью обслуживания s_i (минуты).

Если маршрутный агент прибывает в узел v_i в момент $\tau_i < a_i$, он ожидает до момента a_i ; если $\tau_i > b_i - s_i$ — обслуживание начать невозможно (нарушение ограничения).

Цель: найти порядок посещения всех узлов, минимизирующий суммарное время маршрута $T = \sum_i t_{ij} + \sum_i w_i$, где $w_i = \max(0, a_i - \tau_i)$ — время ожидания в узле v_i , при соблюдении условий $\tau_i \leq b_i - s_i$ для всех i .

На рисунке 1 приведен пример графа задачи TSPTW с тремя узлами. Ребра графа подписаны временем перемещения d_{ij} (в минутах). Алгоритм NNH-TW выбирает оптимальный порядок посещения: $v_0 \rightarrow v_2 \rightarrow v_1$ — посещение ближайшей допустимой точки на каждом шаге.

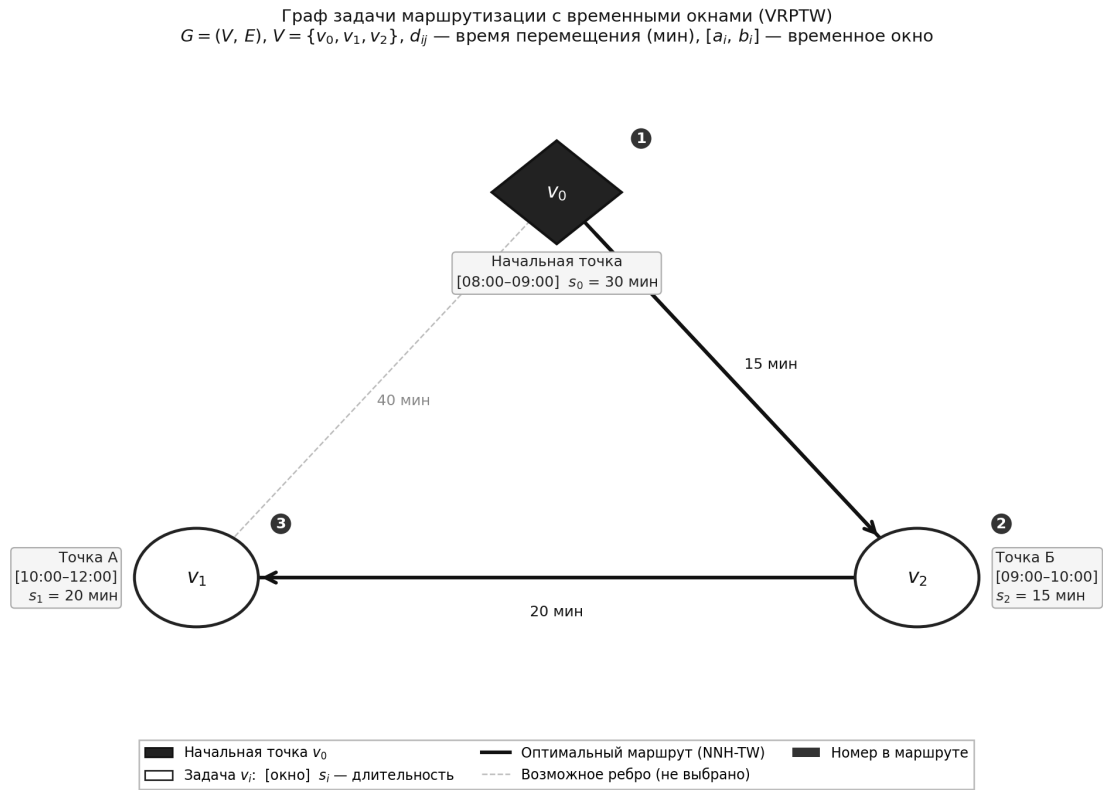


Рисунок 1 – Пример графа задачи коммивояжера с временными окнами (TSPTW): d_{ij} — время перемещения между узлами (мин), $[a_i, b_i]$ — временное окно, s_i — длительность обслуживания; жирными стрелками выделен оптимальный маршрут $v_0 \rightarrow v_2 \rightarrow v_1$

Сравнение алгоритмических подходов. В таблице 4 приведен сравнительный анализ основных подходов к решению TSPTW применительно к задачам данной работы.

Таблица 4 – Сравнение алгоритмических подходов к задаче TSPTW

Подход	Сложность	Оптимальность маршрута
Полный перебор [22]	$O(n!)$	Да
Динамическое программирование (алг. Хелда–Карпа [21])	$O(2^n \cdot n^2)$	Да
Жадная эвристика NNN-TW (ближайший сосед с временными окнами и просмотр вперед)	$O(n^3)$	Нет
Муравьиный алгоритм [26–27]	$O(iter \cdot n^2 \cdot m)$	Нет
Генетические алгоритмы [28]	$O(gen \cdot pop \cdot n)$	Нет

Здесь n — количество узлов маршрута, m — количество муравьев (агентов), $iter$ — число итераций, gen — число поколений, pop — размер популяции.

Обоснование выбора алгоритма. Для веб-приложения, ориентированного на частных пользователей и малый бизнес, был выбран алгоритм «ближайшего соседа с временными окнами» (NNH-TW, Nearest Neighbour Heuristic with Time Windows) по следующим причинам:

1. Соответствие масштабу задачи: система ориентирована на планирование личных задач и курьерских маршрутов малого масштаба, для которых характерно $n < 50$ узлов. Точные методы имеют экспоненциальную сложность и практически неприменимы уже при нескольких десятках узлов, тогда как NNN-TW обладает полиномиальной сложностью $O(n^3)$;

2. Простота реализации: на рассматриваемом масштабе применение простой конструктивной эвристики оправдано. Алгоритм NNN-TW не требует подбора параметров, что снижает сложность реализации, сопровождения и тестирования;

3. Детерминированность: при одних и тех же входных данных алгоритм всегда строит один и тот же маршрут, в отличие от стохастических эвристик, результат которых меняется от запуска к запуску. Воспроизводимость результата обеспечивает предсказуемое поведение интерфейса для пользователя и упрощает автоматизированное тестирование.

1.5 Обзор аналогов и сравнительный анализ

Существующие программные решения, частично пересекающиеся с задачами системы, можно разделить на три класса.

Первый класс — приложения для управления задачами (Todoist [29], Microsoft To Do [30], Singularity [31]),: реализуют полноценное создание, редактирование и удаление задач, поддерживают напоминания и многоплатформенную синхронизацию, однако полностью лишены функций геокодирования адресов и построения маршрутов.

Второй класс — навигационные сервисы (Google Maps [32], Яндекс карты [33], 2GIS [34]): обеспечивают прокладку маршрутов по нескольким точкам, однако не поддерживают автоматическую оптимизацию порядка посещения и не предоставляют средств управления задачами [35].

Третий класс — специализированные логистические платформы (Routific [36], Relog [37], Яндекс Маршрутизация [38], Spoke [39]): сочетают геокодирование, оптимизацию маршрутов и учет временных окон, однако ориентированы на корпоративное управление автопарком (массовая обработка заказов, несколько водителей, диспетчеры) и недоступны для индивидуального применения без коммерческой подписки.

В таблице 5 приведен сравнительный анализ представителей всех трех классов.

Таблица 5 – Сравнительный анализ аналогов

Критерий	Todoist	Microsoft To Do	Google Maps	Routific	Яндекс Маршрутизация	Проектируемое решение
1	2	3	4	5	6	7
Управление задачами	Да	Да	Нет	Нет	Нет	Да
Геокодирование адресов	Нет	Нет	Да	Да	Да	Да
Автоматическая оптимизация маршрута	Нет	Нет	Нет	Да	Да	Да

Продолжение таблицы 5

1	2	3	4	5	6	7
Учет временных окон	Нет	Нет	Нет	Да	Нет	Да
Целевой сегмент	Частные пользователи	Частные пользователи	Массовый	Корпоративный	Средний и крупный бизнес	Частный пользователь и малый бизнес
Импорт/экспорт задач (CSV, XLSX)	Нет	Нет	Нет	Да	Да	Да
Модель распространения	Условно-бесплатная	Бесплатный	Бесплатный	Платная с триалом	Платная подписка	Бесплатный

Сравнение по таблице 5 показывает, что ни один из рассмотренных аналогов не обеспечивает одновременно управление задачами, автоматическую оптимизацию маршрута с учетом временных окон и доступность для частного пользователя без коммерческой подписки. Это подтверждает целесообразность разработки специализированного решения для целевой аудитории.

1.6 Выводы

В первой главе проведен анализ предметной области, выявлены ограничения существующих инструментов и обоснована актуальность разработки системы планирования маршрутов с временными окнами. Сформулированы цель и назначение продукта, определен перечень функциональных и нефункциональных требований.

Сравнительный анализ алгоритмических подходов к задаче TSPTW (полный перебор, динамическое программирование, жадные эвристики, метаэвристики) показал, что для целевого масштаба системы ($n < 50$) и требования по времени ответа наиболее подходящим является алгоритм ближайшего соседа с временными окнами (NNH-TW) с полиномиальной сложностью $O(n^3)$.

На основании сравнения технологий выбран стек разработки: Go, PostgreSQL, React/TypeScript, Vite, Яндекс JS API, Docker Compose. Анализ аналогов подтвердил целесообразность создания решения, ориентированного на частных пользователей и малый бизнес.

2 РЕАЛИЗАЦИЯ

2.1 Модель данных

На рисунке 2 представлена логическая модель данных веб-приложения.

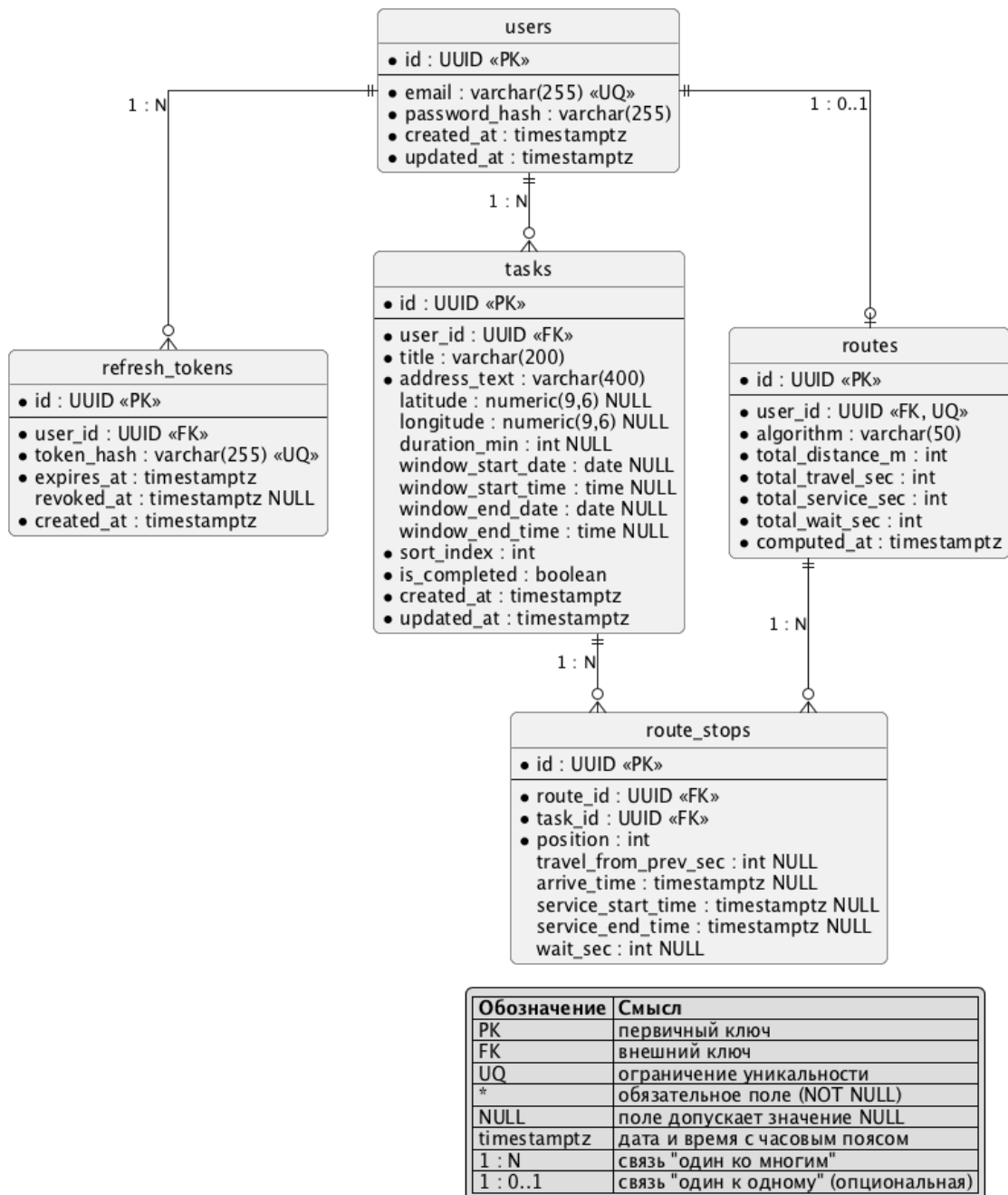


Рисунок 2 – Модель данных

Структура базы данных включает следующие функциональные группы сущностей:

1. хранение информации о пользователях;
2. хранение задач с географической привязкой;
3. хранение маршрутов;
4. хранение результатов вычислений.

Сущности, связанные с пользователями. Сущность `users` предназначена для хранения учетных записей пользователей. Первичный ключ `id` используется для установления связей со всеми зависимыми таблицами. Поле `email` выполняет роль логина и имеет ограничение уникальности, что исключает дублирование учетных записей. Пароль хранится в виде хэшированного значения, что обеспечивает безопасность аутентификации.

Сущность `refresh_tokens` используется для поддержки механизма продления авторизации. Каждый токен связан с конкретным пользователем через внешний ключ `user_id`. Хранение токена в виде хэша предотвращает его компрометацию в случае несанкционированного доступа к базе данных.

Сущность задач. Сущность `tasks` отражает основную предметную сущность системы — задачу, подлежащую выполнению.

Каждая запись содержит:

1. связь с владельцем задачи (`user_id`);
2. текстовое описание адреса (`address_text`);
3. географические координаты (`latitude`, `longitude`), допускающие NULL для задач без адреса;
4. длительность выполнения (`duration_min`), допускающую NULL для мгновенных задач;
5. отдельные дату и время начала (`window_start_date`, `window_start_time`) и окончания (`window_end_date`, `window_end_time`) временного окна, что позволяет задавать ограничения как по дате, так и по времени;
6. индекс сортировки (`sort_index`) для отображения порядка в пользовательском интерфейсе;

7. признак выполнения (`is_completed`) для отметки завершенных задач.

Сущности маршрутов. Сущность `routes` описывает факт построения маршрута для конкретного пользователя. На каждого пользователя хранится не более одного маршрута, что обеспечивается ограничением уникальности `UNIQUE(user_id)`: при повторном запуске оптимизации предыдущая запись и связанные с ней остановки удаляются каскадно. Запись содержит идентификатор использованного алгоритма (`algorithm`), агрегированные показатели маршрута (суммарную длину `total_distance_m`, суммарное время перемещения `total_travel_sec`, суммарное время выполнения задач `total_service_sec` и суммарное ожидание `total_wait_sec`) и момент вычисления `computed_at`. Размещение агрегированных метрик непосредственно в строке маршрута упрощает чтение результатов оптимизации одним запросом и устраняет необходимость отдельной таблицы статистики.

Структура маршрута детализируется в сущности `route_stops`, где каждая запись соответствует отдельной точке маршрута. Поле `position` определяет порядок следования, а временные поля (`arrive_time`, `service_start_time`, `service_end_time`) позволяют зафиксировать результат работы алгоритма с учетом временных окон. Каскадное удаление по внешним ключам `route_id` и `task_id` гарантирует согласованность данных при пересчете маршрута или удалении задачи.

Геометрия маршрута. Геометрическое представление маршрута (полилинии участков между остановками) на сервере не хранится: оно строится клиентской частью с использованием Yandex Maps JS API на основе координат остановок и выбранного режима передвижения. Такой подход исключает необходимость отдельной таблицы для геометрии и обеспечивает актуальность визуализации при изменении транспортного режима без обращения к серверу.

2.2 Архитектура системы

Состав и взаимодействие компонентов. Система построена по клиент-серверной модели и состоит из трех развертываемых компонентов: клиентского одностраничного приложения (SPA) на React, серверного REST API на Go и реляционной базы данных PostgreSQL. Расчет матрицы расстояний и отрисовка карты выполняются через внешний сервис Yandex Maps JS API, а резервный расчет матрицы — через сервис OSRM. Общая структура контейнеров и связи между ними в нотации C4 (уровень контейнеров) приведены на рисунке 3.

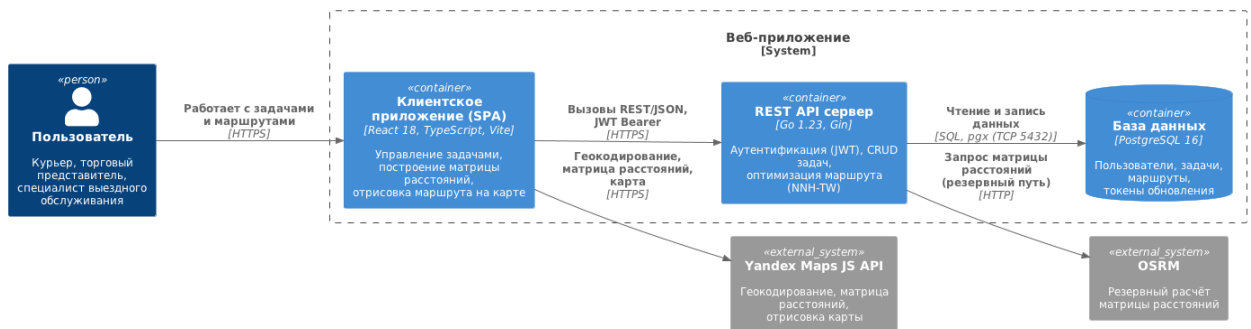


Рисунок 3 – Диаграмма контейнеров системы в нотации C4

Браузер пользователя загружает SPA, которое обращается к серверу по протоколу HTTPS запросами REST; авторизация выполняется по JWT токену. Сервер обращается к PostgreSQL и инкапсулирует бизнес-логику: аутентификацию, управление задачами и оптимизацию маршрута. Матрица расстояний строится преимущественно на стороне клиента средствами Yandex Maps JS API, поскольку бесплатная маршрутизация Яндекса доступна только из JavaScript; если клиент не передал готовую матрицу — например, при недоступности Яндекса после повторных попыток или при обращении к API в обход веб-клиента, — сервер рассчитывает ее самостоятельно через OSRM. Такое разделение ответственности минимизирует серверные обращения к платным сервисам и переносит интерактивную работу с картой на клиента.

Общие принципы. Серверная часть построена по принципам чистой архитектуры, предложенной Р. Мартином [4]. Зависимости направлены от

внешних слоев (HTTP-обработчики, репозитории) к внутренним (доменные сущности и варианты использования), но не наоборот. Такое разделение делает бизнес-логику независимой от конкретного HTTP-фреймворка, СУБД и сторонних сервисов.

Управление задачами. Ядро прикладной логики составляет работа со списком задач пользователя — именно сформированный список служит входными данными для алгоритма оптимизации маршрута. Доменная сущность задачи содержит название, текстовый адрес и географические координаты, длительность выполнения, временное окно (раздельные дата и время начала и окончания) и порядковый индекс в списке. Варианты использования реализуют полный набор операций над списком: создание и редактирование задачи, удаление, получение перечня, пакетное создание, импорт из файла и изменение порядка следования. Доступ к данным инкапсулирован в репозитории поверх PostgreSQL. Каждая операция выполняется в контексте аутентифицированного пользователя, что изолирует списки задач разных пользователей друг от друга.

Кэширование внешних запросов. Обращения к внешним сервисам маршрутизации кэшируются, чтобы при повторной оптимизации одного и того же набора задач не выполнять одинаковые запросы заново. Кэш разделен на два независимых уровня — клиентский и серверный, что обусловлено способом доступа к каждому из сервисов маршрутизации.

Клиентский кэш. Поскольку матрица расстояний строится в браузере, кэш рассчитанных ребер также размещен на стороне клиента — в оперативной памяти в пределах пользовательской сессии. В качестве хранилища применяется LRU-кэш с ограничением размера и временем жизни записи 24 часа. Стратегия LRU выбрана как одна из наиболее эффективных и наименее ресурсоемких политик вытеснения для кэшей с ограниченным объемом [43]. Перед обращением к Yandex API матрица заполняется из кэша, и во внешний сервис отправляются только отсутствующие пары точек, после чего расчи-

танные значения сохраняются обратно. Ключом записи служит режим передвижения и упорядоченная пара координат, округленных до пятого знака после запятой; сравнение выполняется по строковому представлению ключа, что исключает влияние погрешности вычислений с плавающей точкой и незначительных колебаний координат, возвращаемых геокодером.

Серверный кэш. Результаты обращений к серверному сервису маршрутизации OSRM (резервный путь, когда клиент не передал готовую матрицу) кэшируются на уровне приложения в оперативной памяти через обертку `CachedProvider` над интерфейсом `distance.Provider`. В качестве хранилища используется библиотека `hashicorp/golang-lru/v2` [42] (пакет `expirable`) — потокобезопасный LRU-кэш с ограничением размера и временем жизни записи (TTL). Ключом записи служит пара географических координат, округленных до шестого знака после запятой, что делает кэш инвариантным к порядку запросов одних и тех же адресов.

Оба уровня кэша используют политику вытеснения LRU и снижают число обращений к внешним сервисам при повторной оптимизации.

2.3 Алгоритм оптимизации маршрута

Описание алгоритма. Для вычисления оптимального порядка посещения задач реализован алгоритм «ближайшего соседа с временными окнами» (Nearest Neighbour Heuristic with Time Windows, NNH-TW). Базовый алгоритм ближайшего соседа — одна из наиболее изучаемых конструктивных эвристик для задачи коммивояжера, традиционно используемая в качестве эталона при сравнении более сложных методов [44].

Алгоритм является жадной конструктивной эвристикой с одношаговым просмотром вперед: на каждом шаге из допустимых кандидатов выбирается узел с наименьшим временем завершения обслуживания, при этом предпочтение отдается «безопасным» кандидатам — тем, посещение кото-

рых не делает недостижимым ни один другой узел с временным окном. Внутри каждого класса приоритет получает кандидат с более ранним дедлайном, что снижает вероятность пропуска срочных задач. Алгоритм также учитывает дополнительные ограничения: фиксацию начальной и конечной точки маршрута, а также попарные ограничения порядка посещения.

Псевдокод алгоритма приведен в листинге 2.1.

Листинг 2.1 – Псевдокод алгоритма NNH-TW с просмотром вперед

```

1 Вход: граф  $G(V, E)$ , startTimeUnix, constraints{startIdx,
  → endIdx, precedences}
2 Выход: порядок обхода order[], тайминги[], агрегированная
  → статистика

3 1. prereqs <- buildPrereqs(precedences)
4 2. endIdx <- constraints.endIdx (или -1)
5 3. Если constraints.startIdx задан:
6     cur <- constraints.startIdx
7 Иначе:
8     cur <- узел с наиболее ранним окном среди
  → допустимых
9 4. visited[cur] <- true; order <- [cur]
10 5. currentTimeSec <- startTimeUnix + duration[cur] * 60

11 6. Пока |order| < |V|:
12     // Если остался только закрепленный конечный узел –
  → ставим его
13     а. Если endIdx >= 0 И все прочие посещены:
14         Добавить endIdx в маршрут; выйти из цикла

15     // Проход 1: допустимый узел с минимальным
  → completionTime + look-ahead
16     б. bestComp <- +inf; bestSafe <- false;
  → bestDeadline <- +inf; next <- -1
17     в. Для каждого i not in visited, i != endIdx:
18         Если не выполнены предшественники i: пропустить
19         arrivalSec <- currentTimeSec + travelSec[cur][i]
20         Если НЕ feasible(i, arrivalSec): пропустить
21         comp <- completionTime(i, arrivalSec)
22         safe <- НЕ causesWindowMiss(i, comp, G, visited,
  → endIdx)
23         Если (safe > bestSafe) ИЛИ (safe = bestSafe И
  → comp < bestComp)
24             ИЛИ (safe = bestSafe И comp = bestComp И
  → deadline[i] < bestDeadline):

```

```

25         next <- i; bestComp <- comp; bestSafe <-
           ↪ safe

26     // Проход 2: резервный — минимальный completionTime
           ↪ без учета окна
27     d. Если next = -1:
28         Для каждого i not in visited, i != endIdx:
29             Если не выполнены предшественники i:
30                 ↪ пропустить
31             comp <- completionTime(i, arrivalSec)
32             Если comp < bestComp: next <- i

33     e. waitSec <- max(0, windowStart[next] - arrivalSec)
34     f. currentTimeSec <- arrivalSec + waitSec +
           ↪ duration[next] * 60
35     g. visited[next] <- true; order.append(next); cur
           ↪ <- next

36 7. Вернуть order, тайминги и статистику

37 feasible(i, arrivalSec):
38     Если windowEnd[i] < 0: вернуть true
39     Вернуть arrivalSec <= windowEnd[i] - duration[i] * 60

40 causesWindowMiss(cand, afterCandSec, G, visited, endIdx):
41     Для каждого j not in visited, j != cand, j != endIdx:
42         Если windowEnd[j] < 0: пропустить
43         Если НЕ feasible(j, afterCandSec +
44             ↪ travelSec[cand][j]):
45             Вернуть true
46     Вернуть false

```

Блок-схема алгоритма разбита на два рисунка. На рисунке 4 показан основной цикл построения маршрута. Финишная задача и обязательный порядок выполнения (когда одна задача должна предшествовать другой) задаются пользователем как ограничения. Каждая задача может иметь временное окно — интервал, в который ее допустимо выполнить. Стартовую задачу пользователь также может указать явно, иначе она выбирается эвристически — задача без предшественников с наиболее ранним временным окном. Финишная задача, если она указана, резервируется и обслуживается последней: она исключается из обычного выбора и добавляется в конец, когда все прочие задачи уже размещены. На рисунке 5 приведена подпрограмма выбо-

ра очередной задачи. Проход 1 с просмотром вперед берет задачу, до которой успеваем в ее временное окно и все обязательные предшественники которой уже выполнены, отдавая предпочтение той, после которой остальные задачи все еще успевают в свои окна и которая освобождает исполнителя раньше. Резервный проход 2 включается, если в проходе 1 подходящих задач не нашлось, и берет ближайшую по времени завершения задачу без учета временных окон, но также соблюдая обязательный порядок выполнения.

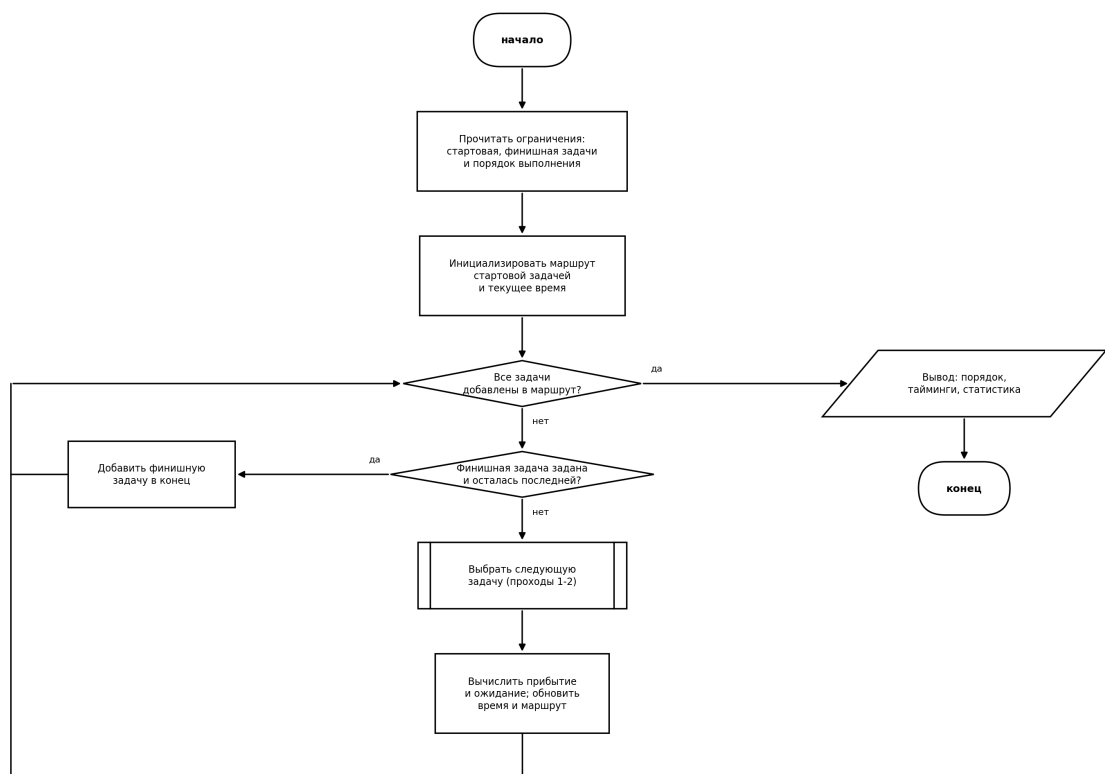


Рисунок 4 – Блок-схема алгоритма NNT-TW:
основной цикл построения маршрута

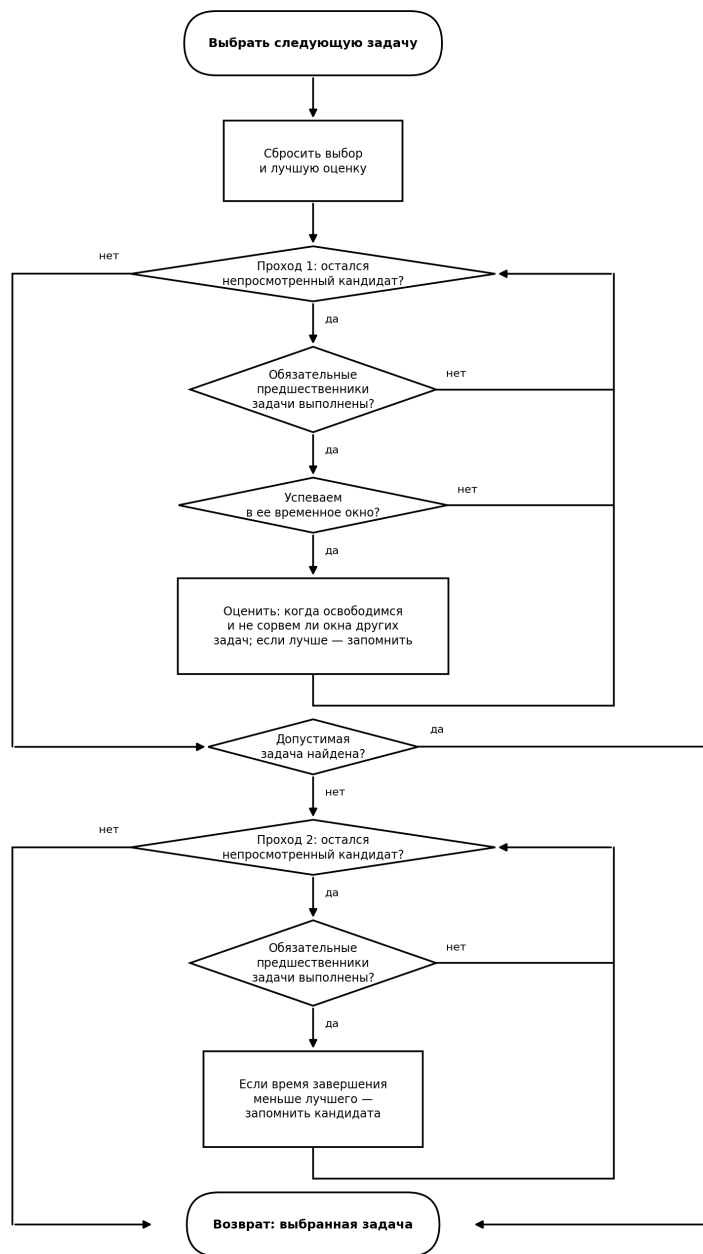


Рисунок 5 – Блок-схема подпрограммы выбора следующей задачи

Трассировка на числовом примере. Ниже приведен пример с тремя задачами для иллюстрации работы алгоритма. Исходные данные приведены в таблице 6, матрица времен перемещения — в таблице 7.

Таблица 6 – Исходные данные задач для трассировки

Узел	Адрес	Временное окно	Длительность
v_0	Офис (начало)	08:00–09:00	30 мин
v_1	Точка А	10:00–12:00	20 мин
v_2	Точка Б	09:00–10:00	15 мин

Таблица 7 – Матрица времен перемещения (минуты)

Из / В	v_0	v_1	v_2
v_0	—	40	15
v_1	40	—	20
v_2	15	20	—

Трассировка алгоритма при начале работы в 08:00 (для наглядности время приведено в минутах от полуночи; в реализации используются секунды Unix):

Выбор стартового узла. Узел v_0 имеет наиболее раннее окно (08:00), поэтому именно он выбирается стартовым: $\text{cur} = 0$, $\text{currentTime} = 480 + 30 = 510$ (08:30 плюс 30 мин выполнения, что соответствует 09:30 или 510 мин от полуночи).

Шаг 1. Проход 1. Для кандидата v_1 прибытие составляет $510 + 40 = 550$ (09:10). Окно v_1 задано как 600–720 (10:00–12:00), условие $550 \leq 720 - 20 = 700$ выполнено, время в пути $t = 40$ мин. Для кандидата v_2 прибытие составляет $510 + 15 = 525$ (08:45), окно 540–600 (09:00–10:00), условие $525 \leq 600 - 15 = 585$ выполнено, $t = 15$ мин. Поскольку v_2 ближе ($15 < 40$), выбирается $\text{next} = 2$. Ожидание открытия окна составляет $\max(0, 540 - 525) = 15$ мин, что дает $\text{currentTime} = 525 + 15 + 15 = 555$ (09:15).

Шаг 2. Проход 1. Для оставшегося кандидата v_1 прибытие составляет $555 + 20 = 575$ (09:35), условие $575 \leq 700$ выполнено, $\text{next} = 1$. Ожидание равно $\max(0, 600 - 575) = 25$ мин, после чего $\text{currentTime} = 575 + 25 + 20 = 620$ (10:20).

Итоговый порядок посещения узлов: $v_0 \rightarrow v_2 \rightarrow v_1$. Суммарное ожидание открытия временных окон по маршруту составляет $15 + 25 = 40$ мин. Полученная последовательность соответствует очередности, при которой ни одно из заданных временных окон не нарушается, а суммарное время в пути и время ожидания минимизированы.

Оценка вычислительной сложности. Алгоритм выполняет n итераций основного цикла; на каждой итерации для каждого из n кандидатов выполняется одношаговый просмотр вперед (causesWindowMiss) по n оставшимся узлам, что дает сложность $O(n^3)$. При $n = 30$ задачах это порядка $30^3 = 27\,000$ элементарных операций. Экспериментальные замеры времени работы алгоритма и его сопоставление с задержкой внешнего API приведены в третьей главе (таблица 9).

Дополнительные ограничения маршрута. Помимо временных окон, реализована поддержка трех типов дополнительных ограничений, задаваемых пользователем при запросе оптимизации.

Фиксированная начальная точка. Если пользователь указывает идентификатор начальной задачи, алгоритм начинает обход с соответствующего узла вместо автоматически выбранного по признаку наиболее раннего временного окна.

Фиксированная конечная точка. Если задана конечная задача, соответствующий узел исключается из стандартного процесса выбора следующей точки. После того как все остальные узлы посещены, конечный узел добавляется в маршрут последним.

Ограничения порядка. Пользователь вправе указать пары задач вида «А до В»: задача А должна быть выполнена строго раньше задачи В. При выборе следующего узла на каждом шаге алгоритм пропускает кандидатов, для которых не все предшественники уже посещены. Условие проверяется в обоих проходах (Pass 1 и Pass 2), что гарантирует соблюдение ограничения независимо от наличия допустимых временных окон.

Ограничения передаются в алгоритм через структуру Constraints пакета optimizer. Преобразование идентификаторов задач в индексы графа выполняется на уровне бизнес-логики (story.Optimize) и не затрагивает реализацию алгоритма, что соответствует принципу единственной ответственности.

2.4 Интерфейс

На рисунках 6, 7 и 8 приведены экранные формы реализованного приложения. Рисунок 6 соответствует странице авторизации, рисунок 7 — главной странице до запуска оптимизации, рисунок 8 — ей же после построения маршрута.

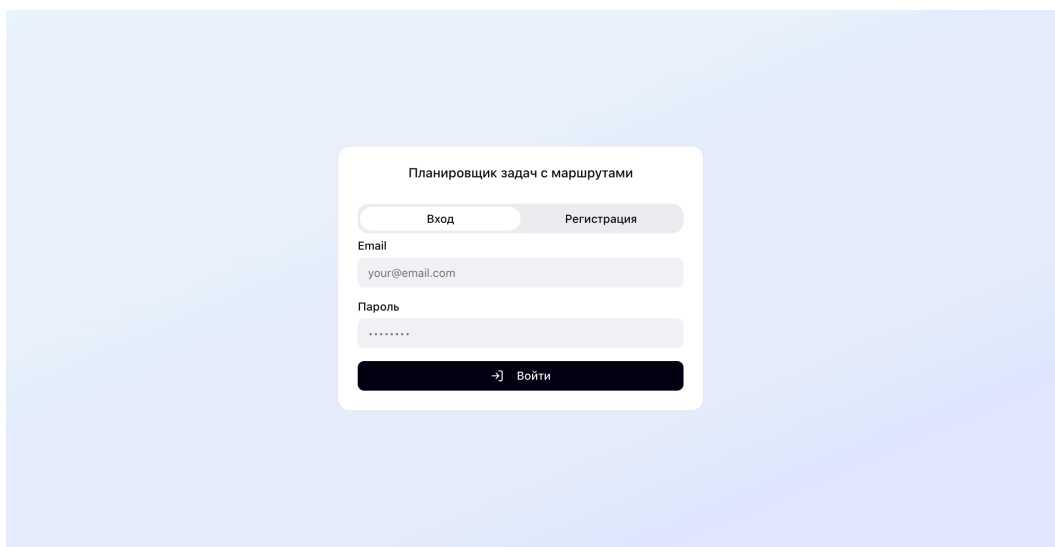


Рисунок 6 – Страница авторизации приложения

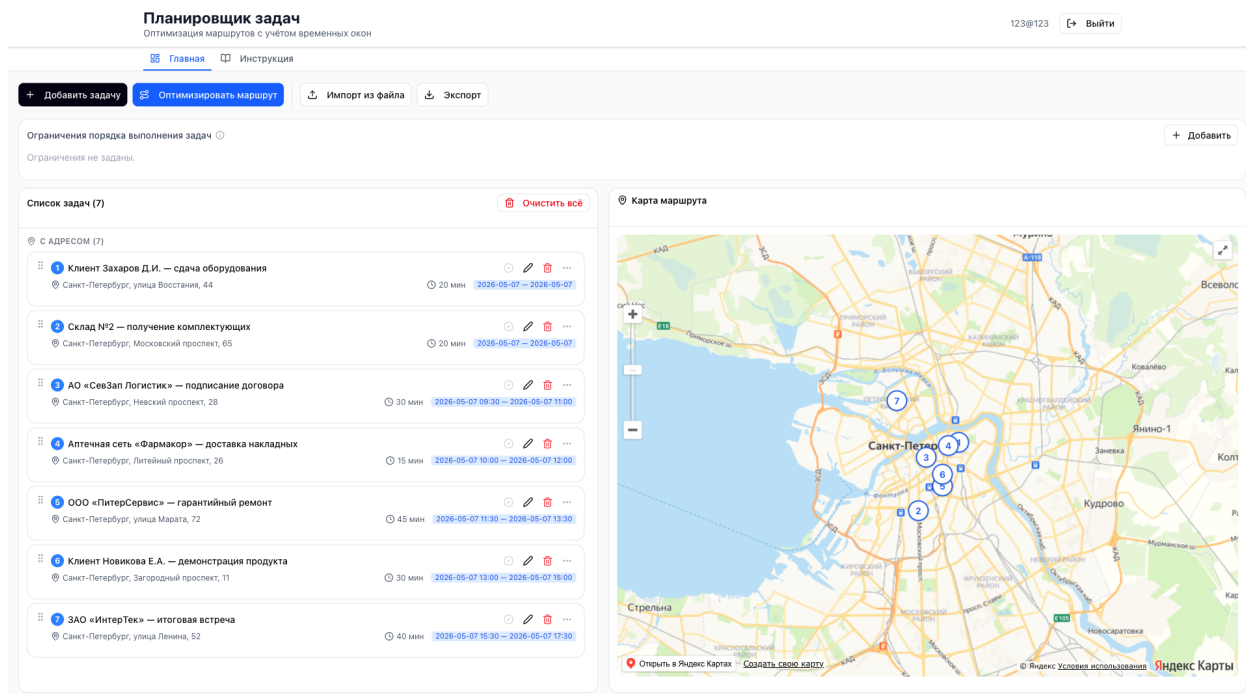


Рисунок 7 – Главная страница приложения до запуска оптимизации

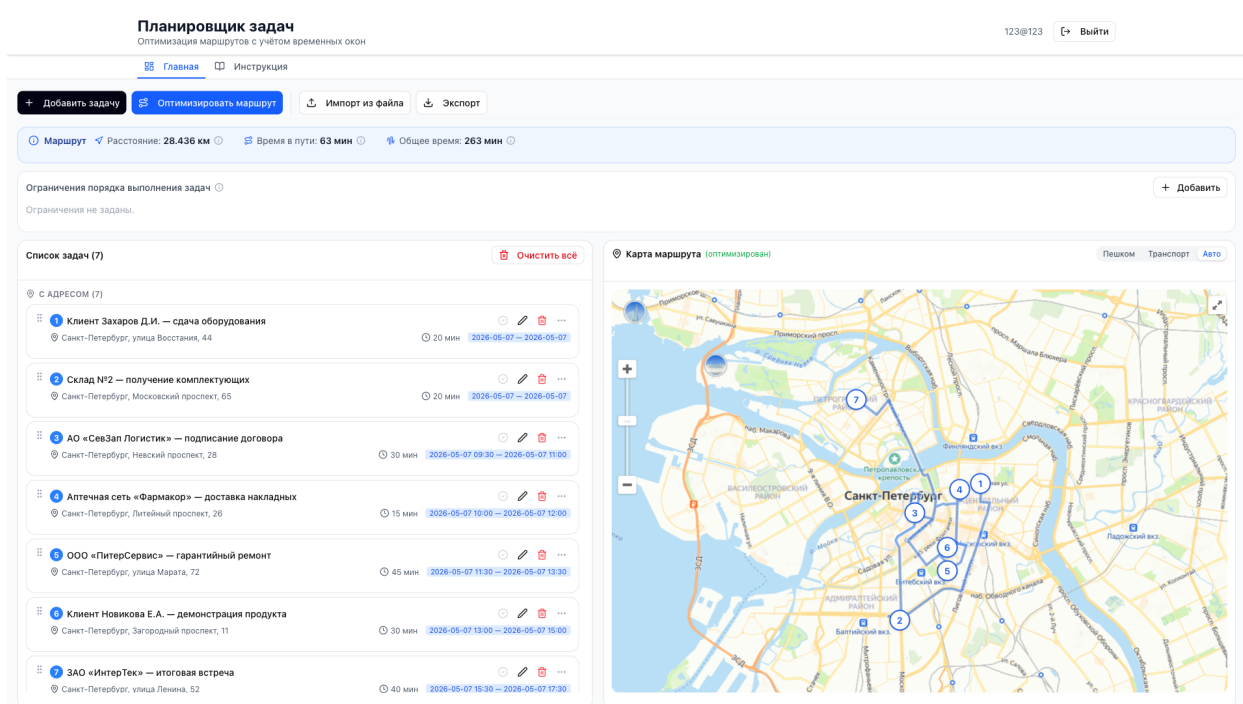


Рисунок 8 – Главная страница после построения маршрута

Структура и организация. Главная страница разделена на две функциональные зоны (рисунок 7): слева — список задач, справа — карта с визуализацией маршрута. Размещение в пределах одного экрана позволяет немедленно оценивать влияние изменений в списке задач на итоговый маршрут без переключения между представлениями, что важно для пользователей без специальной логистической подготовки. Список задач отображается в виде отдельных карточек с фиксированным набором полей (номер, название, адрес, длительность, временное окно). Карта реализована на платформе Яндекс Карты, что обеспечивает связь между порядком задач и их пространственным расположением.

После запуска оптимизации (рисунок 8) над списком появляется панель сводки маршрута с суммарным расстоянием, временем в пути и общим временем выполнения; на карте — линия маршрута с пронумерованными метками и переключатель способа передвижения; карточки перестраиваются в порядке посещения, рассчитанном алгоритмом.

Добавление задачи выполняется через модальное окно с логически сгруппированными полями: название, адрес, длительность, временное окно

(дата и время начала, дата и время окончания) и роль задачи в маршруте (рисунок 9). Обязательно только название; адрес, длительность и окно необязательны, что позволяет вводить как полноценные точки маршрута, так и задачи без географической привязки; минимизация числа обязательных полей снижает порог входа для частных пользователей без специальной подготовки. Поле адреса поддерживает автодополнение через сервис геокодирования Яндекса: по мере ввода под полем отображается список вариантов, выбор которого подставляет координаты задачи. Кнопки «Начальная точка» и «Конечная точка» позволяют сразу при создании закрепить задачу как старт или финиш маршрута.

Добавить задачу ×

Укажите детали задачи. Адрес необязателен — задачи без адреса отображаются отдельно и не включаются в маршрут.

Название задачи *

Встреча с клиентом «Балтийский»

Адрес (необязательно)

Санкт-Петербург, Невский проспект, 28

📍 Россия, Санкт-Петербург, Невский проспект, 28

Длительность (минуты) (необязательно)

Не задана (мгновенная)

Временное окно (необязательно)

Дата и время, в которые должно **начаться** выполнение задачи. Если указана только дата без времени — задача может быть выполнена в течение всего дня. Если дата не указана — используется сегодняшняя.

Начало окна

01.06.2026 📅 × --:-- ⌚

Конец окна

01.06.2026 📅 × --:-- ⌚

Роль в маршруте (необязательно)

☐ Начальная точка ☐ Конечная точка

Отмена Добавить

Рисунок 9 – Модальное окно добавления задачи с автодополнением адреса

Навигация организована через верхнюю панель и включает две вкладки — «Главная» и «Инструкция»; в правой части шапки размещены идентификатор активной учетной записи и кнопка выхода из сессии.

Импорт и экспорт задач. Для ускорения ввода большого количества задач реализован импорт из файлов CSV и Excel (XLSX). Диалог импорта содержит описание ожидаемого формата (столбцы названия, адреса, длительности, даты и границ временного окна) и кнопки загрузки шаблона, а после выбора файла строит предпросмотр распознанных строк (рисунок 10). Каждая строка проверяется: подсчитывается число корректных и ошибочных записей, а строки с нарушениями (пустое название, нечисловая длительность, неверный формат времени, конец окна раньше начала) подсвечиваются красным с пояснением причины. Строки с ошибками пропускаются при сохранении, корректные — импортируются и геокодируются автоматически, что исключает попадание некорректных данных в список задач.

Предпросмотр (5 строк) 🟢 Корректных: 4 🟡 С ошибками: 1

#	Название	Адрес	Длит., мин	Дата	Окно	Статус
1	Подписание акта	Санкт-Петербург, Не...	30	2026-05-31	09:30 – 11:00	🟢
2	Доставка образцов	Санкт-Петербург, Ли...	15	2026-05-31	10:00 – 12:00	🟢
3	Осмотр объекта	Санкт-Петербург, ул...	45	2026-05-31	11:30 – 13:30	🟢
4	Звонок поставщику	—	10	2026-06-01	—	🟢
5	Проверка склада	Санкт-Петербург, За...	NaN	2026-05-31	14:00 – 12:00	✖ duration должно быть целым числом ≥ 1; window_end должен быть позже window_start

Строки с ошибками будут пропущены. Исправьте файл и загрузите повторно.

Рисунок 10 – Предпросмотр импортируемых задач с проверкой строк на ошибки

Экспорт доступен в трех форматах. В CSV Excel (XLSX) выгружается список задач пользователя с полями: название, адрес, длительность, дата и время временного окна; в JSON — маршрут целиком, с координатами точек, порядком посещения и сводными показателями (суммарные расстояние и время).

Дополнительные элементы управления. В интерфейсе реализована возможность изменения порядка задач методом перетаскивания: даже после автоматической оптимизации пользователь может скорректировать последовательность вручную без повторного запуска алгоритма. Помимо этого, реализованы перечисленные ниже элементы управления.

Выбор режима передвижения. Над картой размещен переключатель способа перемещения — пешком, общественный транспорт или автомобиль (рисунок 11). Выбранный режим определяет матрицу расстояний, запрашиваемую у Яндекс Карт, и влияет на визуализацию: для автомобильного и пешего маршрута строится единая линия с суммарным временем и расстоянием, а для общественного транспорта — цепочка сегментов с указанием номеров маршрутов и остановок.

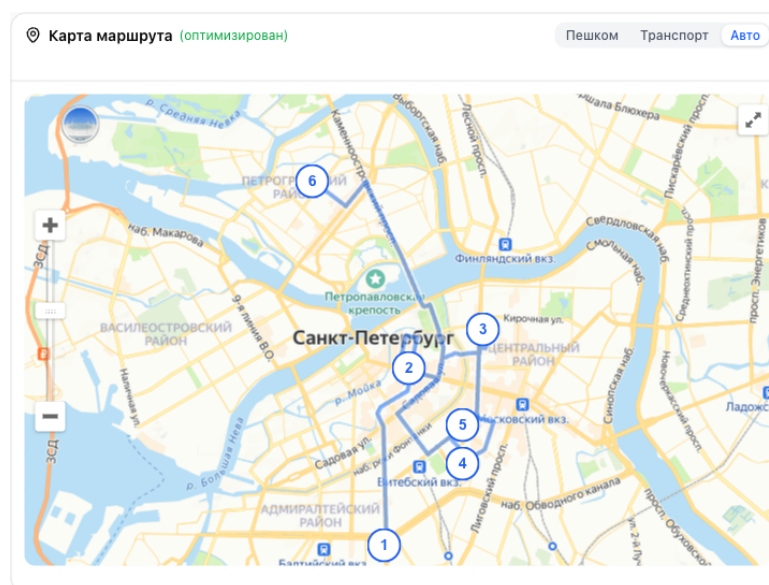


Рисунок 11 – Переключатель режима передвижения и визуализация маршрута на карте

Фиксация начальной и конечной точек. Через контекстное меню карточки задачи пользователь назначает точку старта или финиша маршрута; закрепленные задачи помечаются в списке метками «Старт» и «Финиш» и передаются алгоритму как фиксированные концевые узлы (рисунок 12).

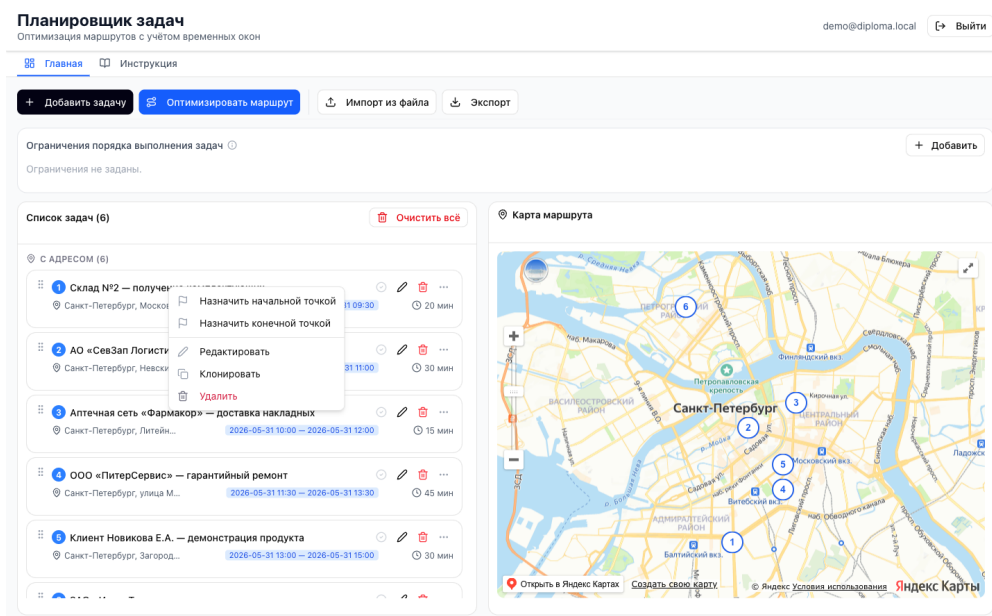


Рисунок 12 – Контекстное меню карточки задачи: назначение старта и финиша, клонирование, редактирование и удаление

Ограничения порядка. Пользователь задает пары «задача А до задачи В» (рисунок 13), которые алгоритм обязан соблюдать при построении маршрута; число таких пар не ограничено.

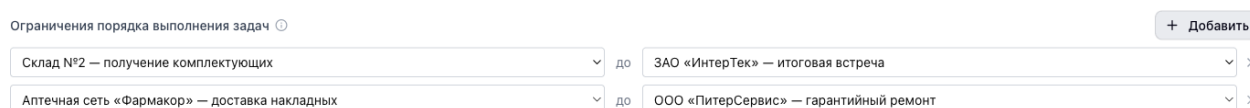


Рисунок 13 – Задание ограничений порядка выполнения задач

Отметка выполнения. Задачу можно пометить как выполненную; такая карточка отображается приглушенным цветом с зачеркнутым названием и исключается из последующей оптимизации.

Клонирование задач. Команда «Клонировать» контекстного меню создает копию задачи с суффиксом «(копия)» и теми же адресом, длительностью и временным окном, что ускоряет ввод однотипных записей.

Обнаружение конфликтов. Интерфейс автоматически выявляет задачи с некорректными или взаимно несовместимыми временными окнами и работает в двух режимах — до и после оптимизации.

До оптимизации проверяются сами окна. Помечаются задачи с инвертированным окном (время окончания не позже времени начала), задачи, длительность обслуживания которых превышает ширину окна, а также группы

задач, чьи окна перекрываются настолько, что суммарное время обслуживания не уместится в общий интервал. Конфликтующие карточки выделяются красной рамкой, красной меткой окна и значком предупреждения (рисунок 14), а кнопка запуска оптимизации блокируется с подсказкой об устранении конфликтов. Тем самым исключается запуск алгоритма для заведомо неразрешимой постановки.

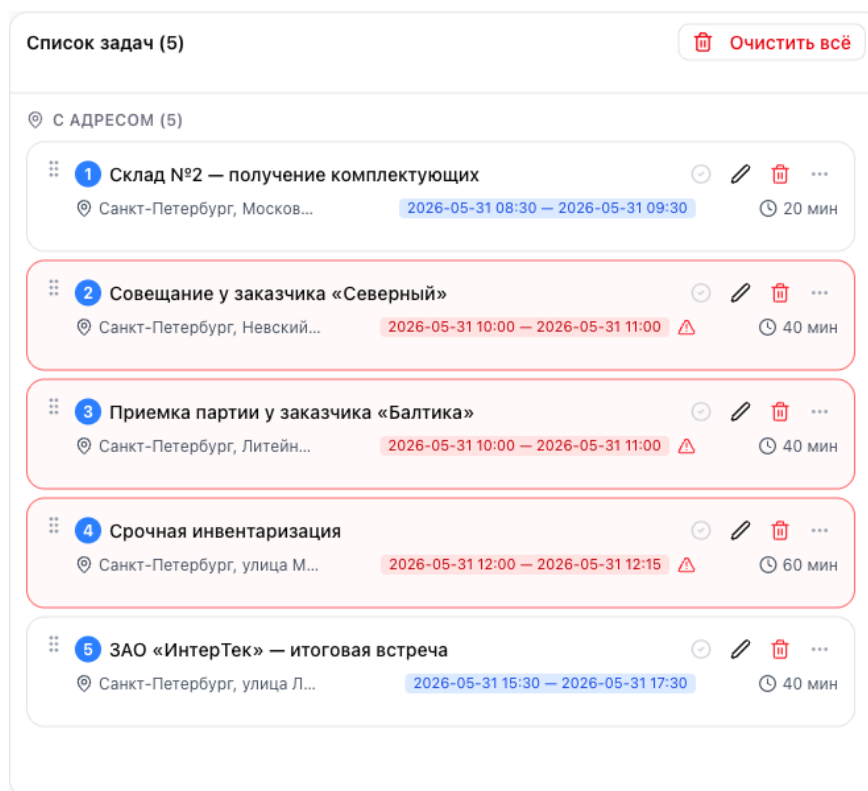


Рисунок 14 – Обнаружение конфликтов временных окон до запуска оптимизации

После оптимизации проверяется фактическое расписание. Для каждой остановки расчетное время прибытия сопоставляется с крайним сроком окна с учетом длительности обслуживания. Если задача не успевает быть обслужена в пределах своего окна, над списком остановок выводится предупреждение о числе таких задач, а соответствующая остановка подсвечивается красным с отметкой «Не укладывается в окно» (рисунок 15). Это позволяет увидеть, что при заданном наборе ограничений отдельные окна физически недостижимы, и скорректировать исходные данные — длительность, границы окна или состав маршрута.

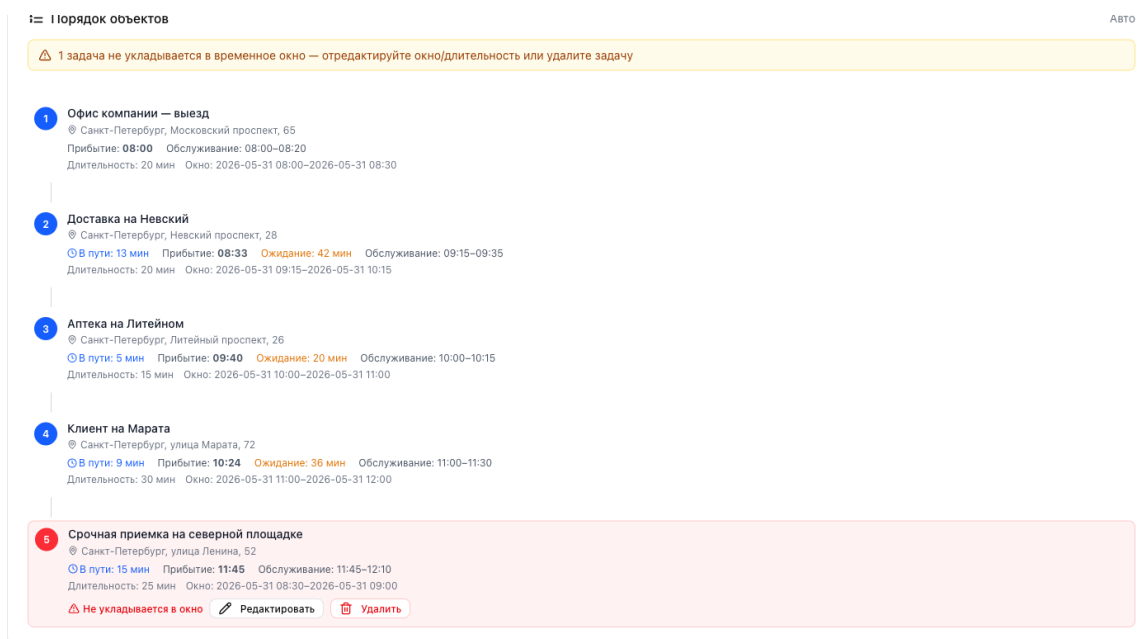


Рисунок 15 – Обнаружение фактического опоздания после построения маршрута

Адаптивная верстка. Интерфейс сохраняет работоспособность на устройствах различных классов: настольные мониторы, планшеты и смартфоны; поддержка смартфонов существенна для курьеров и мастеров на выезде, работающих вне стационарного рабочего места. Адаптация выполнена средствами фреймворка Tailwind CSS [45] с применением контрольных точек ширины.

На широких экранах шириной от 1024 пикселей (рисунок 7) список задач и карта размещаются рядом в две колонки, что обеспечивает их одновременный обзор. При сужении области просмотра до ширины менее 1024 пикселей двухколоночная компоновка перестраивается в одноколоночную: карта сдвигается под список задач, а кнопки верхней панели инструментов переносятся на дополнительные строки. На экранах смартфонов шириной менее 640 пикселей текстовые подписи кнопок панели инструментов сворачиваются до иконок (рисунок 16), а длинные названия задач усекаются с многоточием для сохранения целостности карточек.

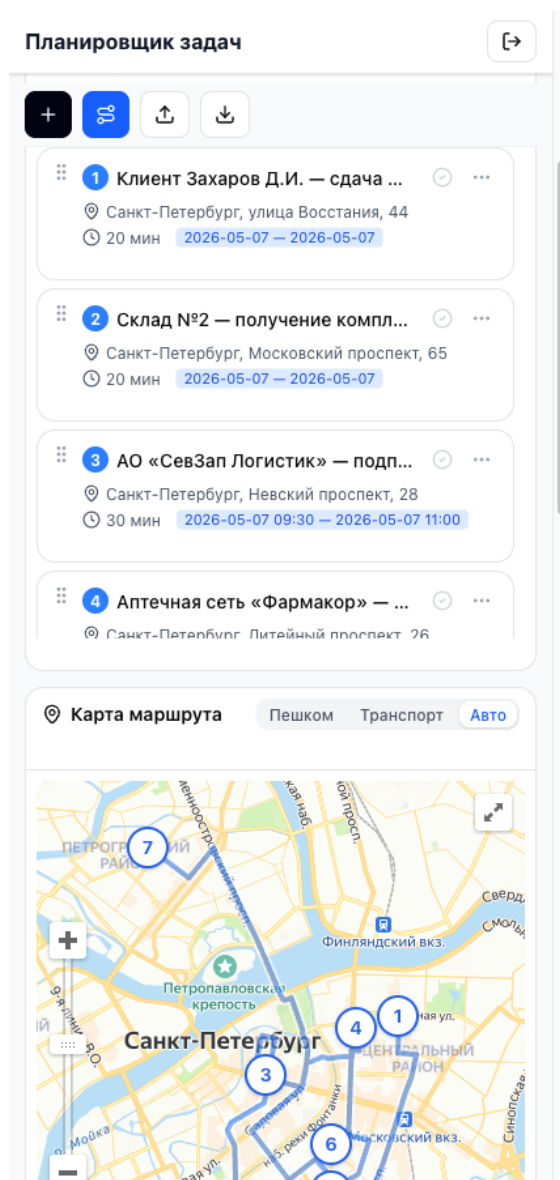


Рисунок 16 – Главная страница приложения на смартфоне после построения маршрута

2.5 Техническая реализация

Состав и объем программного кода. Приложение включает два независимых программных компонента: серверную часть на Go и клиентскую часть на TypeScript с библиотекой React. Серверная часть содержит около 54 файлов Go, организованных по пакетам в соответствии с принципами чистой архитектуры (доменные сущности и варианты использования, HTTP-обработчики, репозитории PostgreSQL, провайдер матрицы расстояний) и SQL-миграцию с финальной схемой базы данных. Клиентская часть вклю-

чает около 100 файлов TypeScript/TSX, из которых около 40 реализуют прикладную логику и компоненты страниц, а остальные — примитивы библиотеки пользовательского интерфейса на основе shadcn/ui [46] и Radix UI [47].

Наиболее значимые фрагменты кода приведены в приложениях: реализация алгоритма оптимизации маршрута — в приложении Б.1, клиент API с логикой обновления токена — в приложении Б.2, утилита построения матрицы расстояний — в приложении Б.3.

REST-API управления задачами. Операции планирования задач доступны клиенту через набор HTTP-эндпоинтов с аутентификацией по JWT токену; тело запросов и ответов передается в формате JSON:

1. GET /api/tasks — получение списка задач пользователя;
2. POST /api/tasks — создание задачи;
3. POST /api/tasks/batch — пакетное создание нескольких задач за один запрос;
4. POST /api/tasks/import — импорт задач из файла формата CSV или XLSX;
5. PATCH /api/tasks/order — изменение порядка следования задач в списке;
6. PATCH /api/tasks/{id} — частичное обновление полей задачи;
7. DELETE /api/tasks/{id} — удаление задачи.

Требования к техническим средствам. Для развертывания и эксплуатации приложения предъявляются следующие требования к техническим средствам.

Для серверной части:

1. процессор: одноядерный с тактовой частотой не ниже 1 ГГц (рекомендуется двухъядерный и более);
2. оперативная память: не менее 512 МБ (рекомендуется 1 ГБ и более);
3. дисковое пространство: не менее 1 ГБ для хранения образов Docker и данных PostgreSQL;

4. сетевой интерфейс: соединение с интернетом для обращения к внешним сервисам (Яндекс Карты, OSRM).

Для клиентской части (рабочее место пользователя):

1. устройство с современным веб-браузером и поддержкой JavaScript;
2. соединение с интернетом для загрузки Яндекс Карт и обращения к API.

Требования к программному обеспечению. Для развертывания серверной части необходимы:

1. операционная система Linux [48], macOS [49] или Windows [50] с поддержкой контейнеризации;
2. Docker Engine версии 24.0 и выше;
3. Docker Compose версии 2.0 и выше.

Для сборки и запуска клиентской части в режиме разработки необходимы Node.js [51] версии 18 и выше и менеджер пакетов npm [40] версии 8 и выше (либо pnpm версии 9 и выше). На стороне пользователя поддерживаются браузеры Google Chrome версии 90 и выше, Mozilla Firefox версии 88 и выше, Apple Safari версии 14 и выше, Microsoft Edge версии 90 и выше, а также Яндекс Браузер версии 25 и выше.

2.6 Выводы

Спроектирована модель данных, поддерживающая хранение задач, маршрутов, временных окон и пользовательских ограничений.

Серверная часть построена по принципам чистой архитектуры с разделением слоев домена, вариантов использования и инфраструктуры; реализован алгоритм NNH-TW с одношаговым просмотром вперед и поддержкой фиксированных начальной и конечной точек маршрута, а также попарных ограничений порядка посещения.

Разработан клиентский интерфейс с интерактивной картой и средствами ручного управления маршрутом; обеспечена контейнеризация приложения на основе Docker Compose для воспроизводимого развертывания.

3 ВНЕДРЕНИЕ И ЭКСПЛУАТАЦИЯ

3.1 Анализ внедрения продукта

В первой главе сформулированы цель разработки и перечень функциональных и нефункциональных требований к системе. Сопоставление этих требований с результатами реализации приведено в таблице 8.

Таблица 8 – Соответствие реализованной системы поставленным требованиям

Требование	Результат
1	2
Создание, редактирование и удаление задач	Реализовано
Хранение географических координат	Реализовано
Получение матрицы расстояний через внешние API	Реализовано (Яндекс мультимаршрутизатор на клиенте, OSRM как резервный, кэш в памяти приложения)
Вычисление оптимальной последовательности с учетом временных окон	Реализовано
Расчет времени прибытия, ожидания и завершения	Реализовано
Визуализация маршрута на карте	Реализовано (Яндекс Карты)
Экспорт задач (CSV, XLSX) и маршрута (JSON)	Реализовано
Импорт задач из CSV и XLSX	Реализовано
Дополнительные ограничения маршрута (начальная/конечная точка, порядок)	Реализовано
Выбор режима транспорта	Реализовано (автомобиль, пешком, общественный транспорт; в последнем случае расчет времени приближается автомобильным режимом, визуализация выполняется корректно)
Обнаружение конфликтов временных окон	Реализовано

Продолжение таблицы 8

1	2
Время ответа сервера не более 2 секунд	Выполнено
Поддержка современных браузеров	Выполнено
Устойчивость к повторным запросам API за счет кэширования	Выполнено
Масштабируемость до 100 одновременных пользователей	Подтверждено нагрузочным тестированием: не менее чем 13-кратный запас относительно 100 пользователей при $p95 < 2\,000$ мс и нулевой доле ошибок

Все функциональные и нефункциональные требования, поставленные в первой главе, выполнены. Корректность реализации подтверждена модульными и интеграционными тестами, а выполнение требований к производительности и масштабируемости — нагрузочным тестированием.

3.2 Аспекты отказоустойчивости, надежности и производительности

Устойчивость к отказам внешних зависимостей. Ключевым риском при интеграции с внешним картографическим сервисом является его деградация или временная недоступность. На стороне сервера этот риск снижает кэш матрицы расстояний, рассмотренный в разделе «Архитектура системы» второй главы: при наличии записи в кэше повторное обращение к OSRM не производится, что уменьшает суммарную задержку оптимизации и исключает зависимость от мгновенной доступности маршрутизатора.

На стороне клиента загрузка Яндекс JS API выполняется через утилиту `loadYandexMaps()`, которая инжектирует `<script>`-тег динамически при первом обращении и возвращает один и тот же `Promise` при последующих вызовах. Такой подход предотвращает дублирующую загрузку скрипта при

одновременном обращении нескольких компонентов и исключает состояние гонки при инициализации API.

Резервный расчет матрицы через OSRM. Каждый запрос к маршрутизатору Яндекса по паре точек выполняется с повторными попытками (до трех раз с нарастающей задержкой между ними), что сглаживает кратковременные сетевые сбои и единичные отказы сервиса. Если же недоступным оказывается сам Яндекс — скрипт API не загрузился либо все запросы маршрутизации завершились ошибкой после исчерпания повторов, — клиент не передает матрицу в запросе оптимизации, и сервер рассчитывает ее самостоятельно через OSRM. Тем самым устраняется единая точка отказа: оптимизация маршрута остается работоспособной при полной недоступности клиентского картографического сервиса, а пользователь информируется о переключении на резервный источник. Последовательность принятия решения о источнике матрицы приведена на рисунке 17.

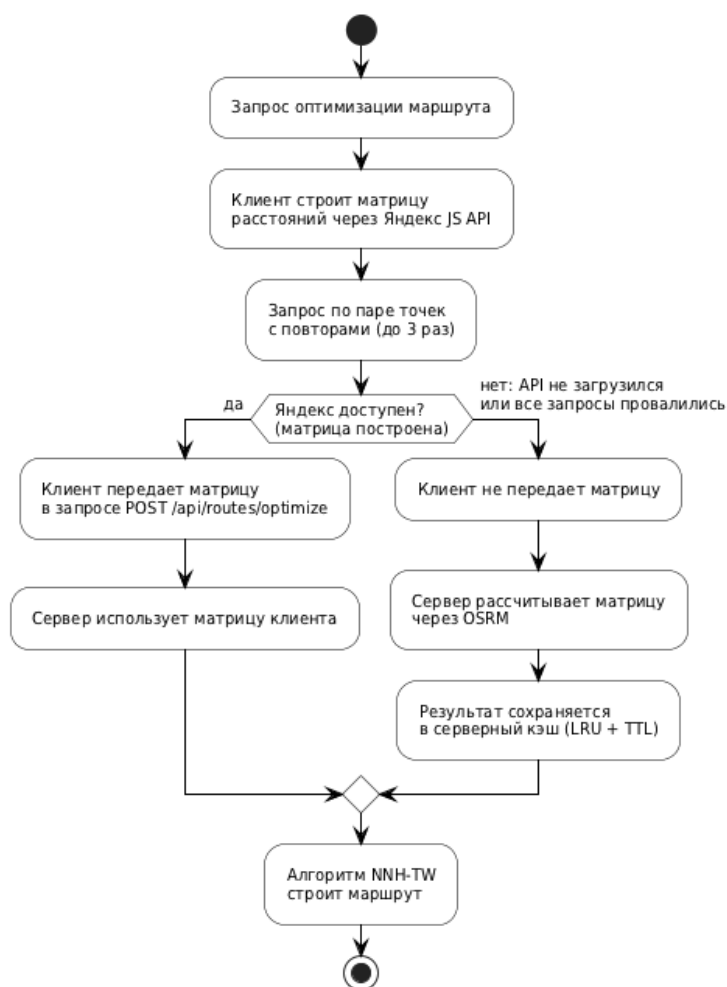


Рисунок 17 – Выбор источника матрицы расстояний:
Яндекс на клиенте или OSRM на сервере

Надежность аутентификации. Аутентификация реализована по схеме JWT [52] с двумя токенами: краткосрочным access-токеном (время жизни — 15 минут) и долгосрочным refresh-токеном (время жизни — 30 дней). Применение пары access/refresh-токенов соответствует общему подходу к делегированной авторизации, описанному в [53]. Токены обновления хранятся в базе данных в виде хэша SHA-256 [54], что исключает их компрометацию при несанкционированном доступе к СУБД; данная мера согласуется с практическими рекомендациями по защите сервисов, использующих JWT [55]. Пароли пользователей хэшируются алгоритмом bcrypt [56] с адаптивным количеством раундов.

На клиентской стороне реализован механизм автоматического обновления токена при получении ответа с кодом 401. Корректность этого меха-

низма проверяется модульными тестами API-клиента с перехватом HTTP-запросов через Mock Service Worker (MSW) [57]. Для исключения множественных одновременных запросов на обновление применяется дедупликация: при первом истечении токена запускается единственный запрос обновления, остальные запросы ожидают его завершения через общий Promise. После успешного обновления исходный запрос повторяется автоматически — пользователь не замечает прерывания сессии.

Сессия пользователя сохраняется в `localStorage` под ключом `planner.auth.session`. При закрытии и повторном открытии браузера приложение автоматически восстанавливает состояние аутентификации без повторного ввода учетных данных.

Диаграммы последовательности, отражающие три сценария взаимодействия компонентов системы аутентификации, представлены на рисунках 18, 19 и 20.

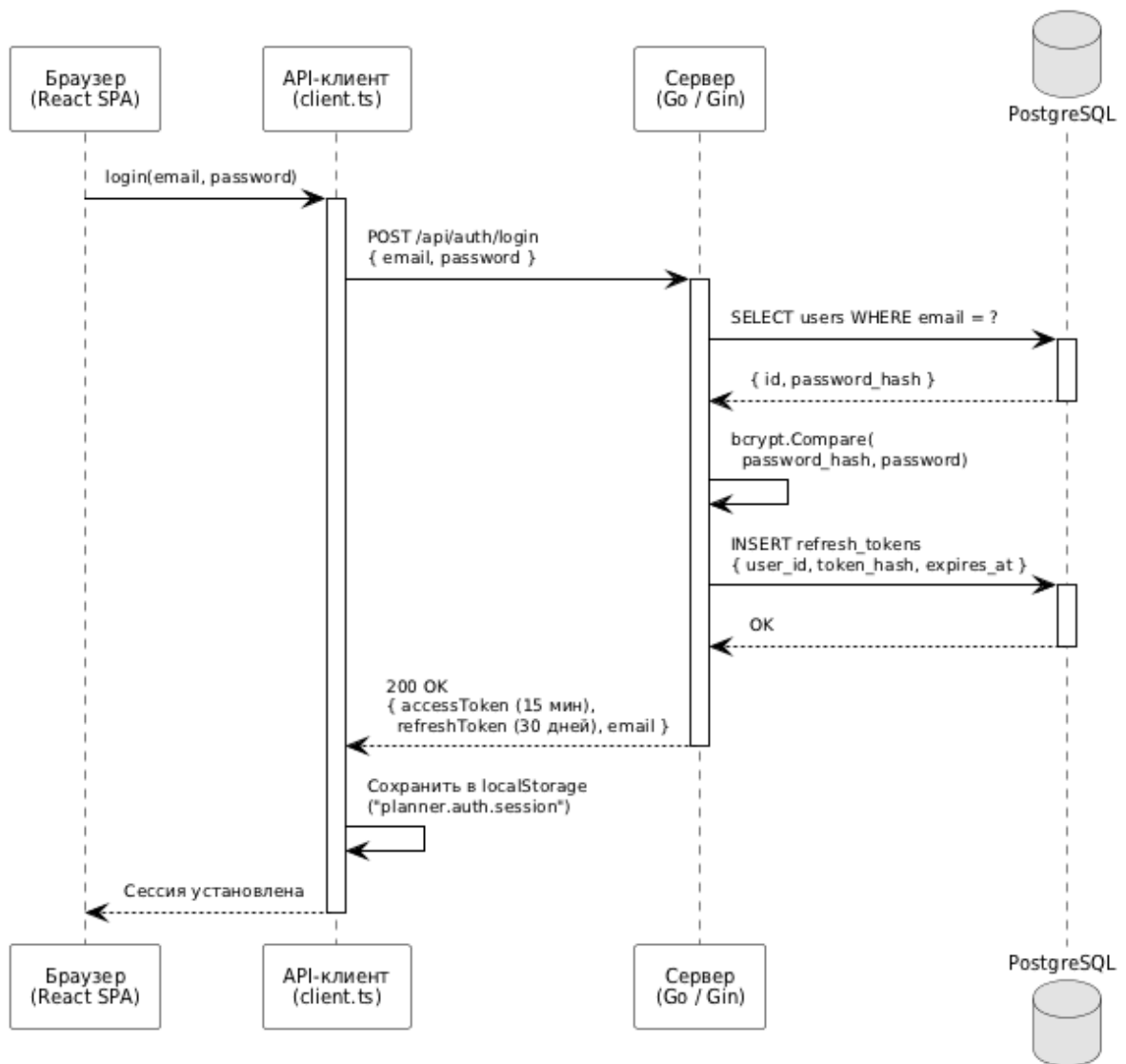


Рисунок 18 – Диаграмма последовательности: сценарий входа в систему

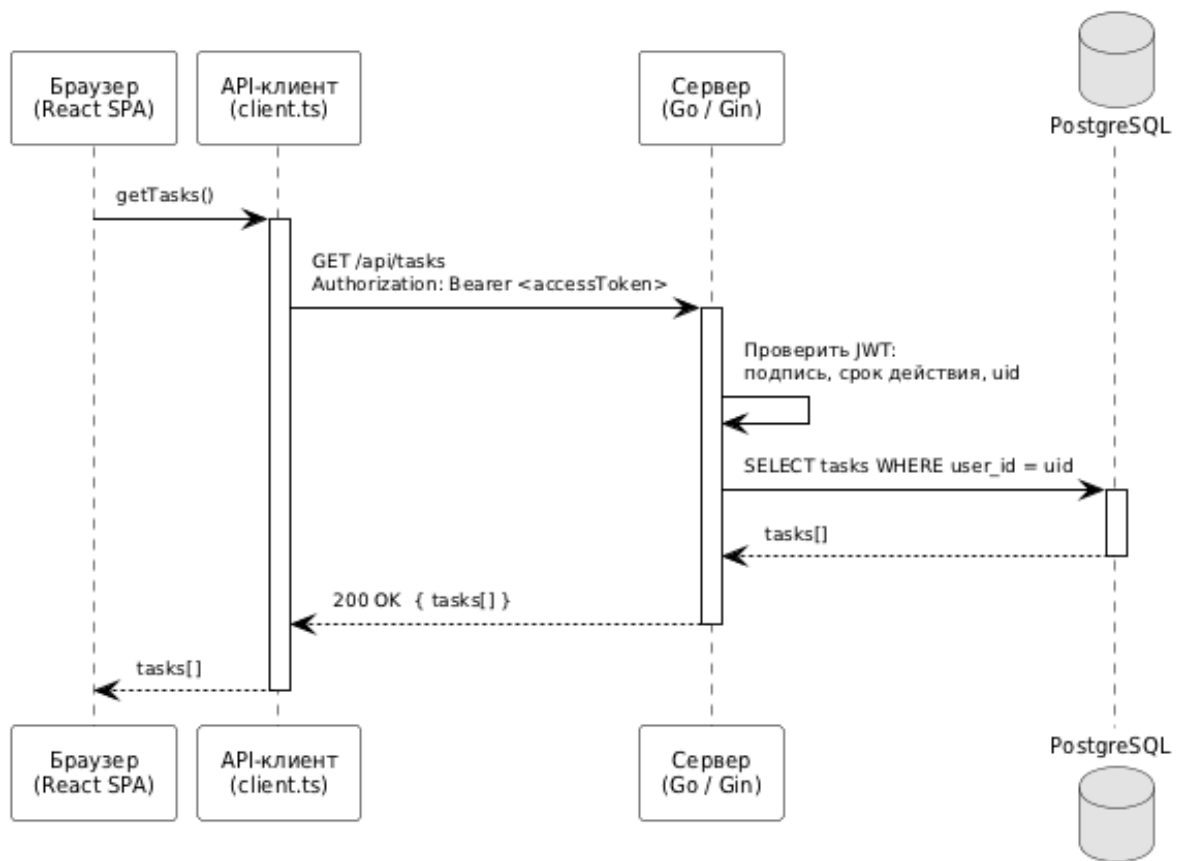


Рисунок 19 – Диаграмма последовательности: запрос с валидным токеном

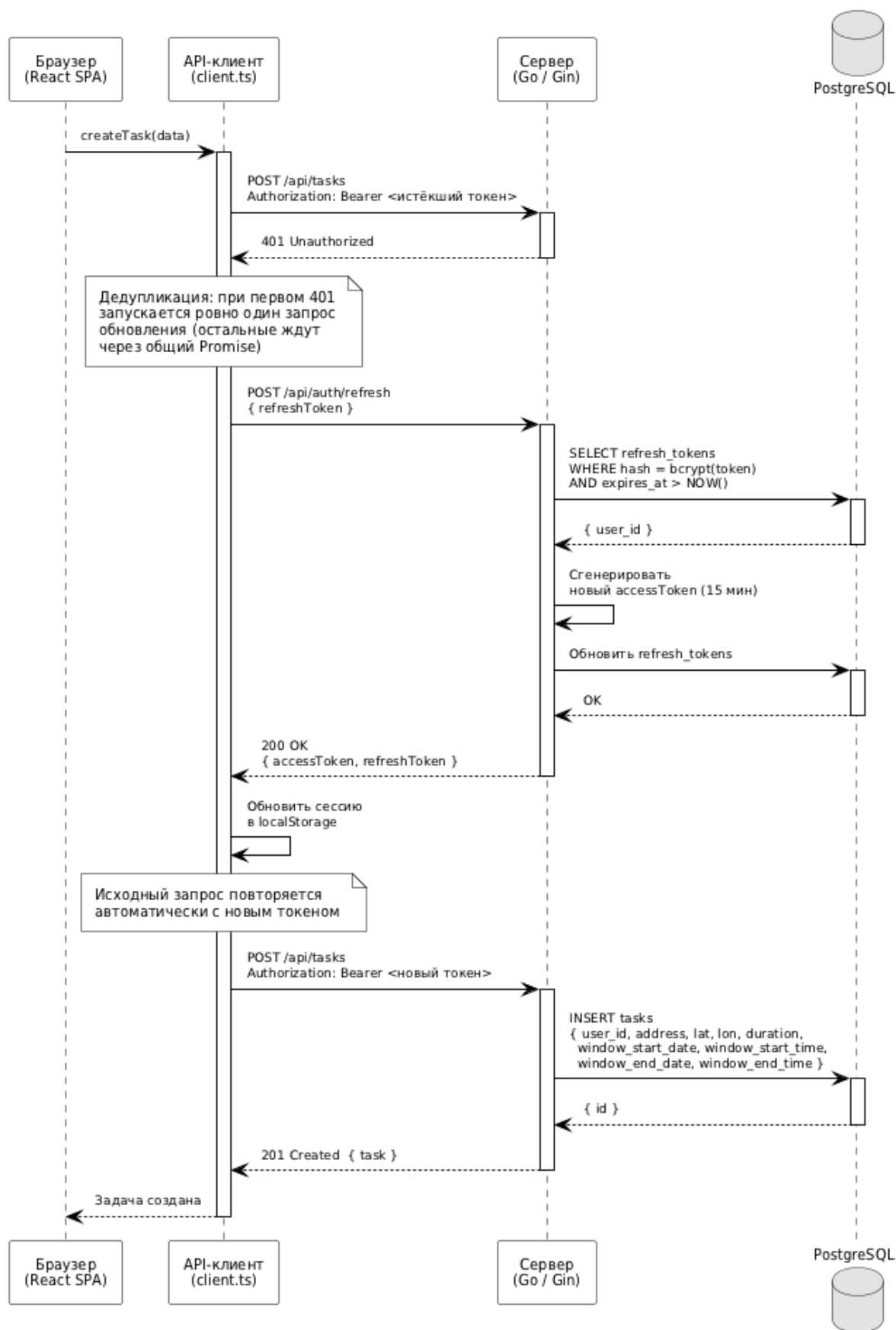


Рисунок 20 – Диаграмма последовательности:
автоматическое обновление токена при ответе 401

Надежность инфраструктуры. При развертывании с использованием Docker Compose [20] зависимость между контейнерами управляется через механизм healthcheck: сервис применения миграций и API-сервер запускаются только после того, как контейнер базы данных PostgreSQL подтвердит готовность к приему соединений командой pg_isready. Это исключает ситуацию, при которой API-сервер стартует до завершения инициализации СУБД.

Схема базы данных управляется инструментом migrate/migrate [58], запускаемым автоматически при каждом старте стека. Применение миграций является идемпотентным: повторный запуск migrate up при уже актуальной схеме не вносит изменений. Это позволяет безопасно перезапускать стек без ручного контроля состояния миграций.

Производительность алгоритма оптимизации. Для оценки производительности алгоритма NNH-TW использован встроенный инструмент бенчмаркинга языка Go (testing.Benchmark). Измерения проводились на рабочей станции с процессором Apple M1 (8 ядер) и 16 ГБ оперативной памяти под управлением операционной системы macOS 15. Для каждого значения n выполнялось не менее 50 000 итераций; в таблице приведено среднее время одного вызова. Результаты представлены в таблице 9.

Таблица 9 – Производительность алгоритма NNH-TW в зависимости от числа задач

Число задач (n)	Время NNH-TW (Go)
5	0,36 мкс
10	1,12 мкс
20	3,66 мкс
30	6,96 мкс
50	24,54 мкс

Из таблицы 9 следует, что узким местом системы является не сам алгоритм оптимизации (сложность $O(n^3)$, выполнение при $n \leq 30$ за единицы-десятки микросекунд), а получение матрицы расстояний от внешнего API. При $n = 20$ задачах число параллельных запросов к Яндекс JS API составляет $20 \times 19 = 380$, однако эти запросы выполняются параллельно на стороне

клиента через Promise.all, что сводит суммарную задержку к времени наиболее медленного из них; доминирующая задержка определяется сетевыми условиями и временем ответа внешнего API.

Нагрузочное тестирование. Производительность REST API проверена по открытой модели нагрузки с помощью инструмента k6 [59]: вместо фиксированного числа пользователей задается целевая интенсивность запросов (RPS, requests per second). Такой подход показывает, как ведет себя конкретная ручка при росте RPS, и позволяет найти точку насыщения — интенсивность, за которой время отклика выходит за допустимый предел или появляются ошибки.

Тестировались три основные ручки по отдельности: чтение списка задач (GET /api/tasks), создание задачи (POST /api/tasks) и построение маршрута (POST /api/routes/optimize). Каждая ручка прогонялась на наборе целевых значений RPS; Тестовые задачи готовились заранее в фазе инициализации, поэтому в измерение попадает ровно один запрос к целевой ручке без аутентификации и сетевых задержек. Для ручки оптимизации матрица расстояний предварительно прогревалась, поэтому в фазе замера она бралась из кэша в памяти приложения и обращения к внешнему OSRM не происходило — измерялась чистая работа алгоритма NNH-TW и СУБД.

Тестовый стенд: ноутбук с процессором Apple M1 (8 ядер), 16 ГБ оперативной памяти; серверный стек (API и PostgreSQL) и генератор нагрузки k6 запущены на одной машине через Docker Compose, поэтому измеренный предел отражает связку «приложение + БД + генератор нагрузки» без сетевых задержек. Критерии успешности для каждой точки: 95-й перцентиль времени ответа менее 2000 мс, доля ошибочных HTTP-ответов менее 1%. Результаты по трем ручкам приведены в таблицах 10–12, а зависимость p95 от RPS показана на рисунке 21. В таблицах столбец «Целевой RPS» — это заданная интенсивность запросов, а «Факт. RPS» — фактически достигнутая за время замера частота обработанных запросов. Пока сервер успевает обрабатывать

поток, эти значения совпадают; если же фактический RPS падает ниже целевого, значит наступило насыщение — система не справляется с входным потоком и часть запросов накапливается в очереди.

Таблица 10 – Нагрузочное тестирование ручки GET /api/tasks (чтение списка задач)

Целевой RPS	Факт. RPS	avg, мс	p95, мс	p99, мс	Ошибки
100	100	1,8	2,5	3,8	0%
400	400	1,5	2,4	7,4	0%
800	800	1,2	2,1	15	0%
1 600	1 600	2,6	6,3	69	0%
3 200	3 200	10	65	223	0%
6 400	3 860	1 479	3 391	3 822	0%
12 800	2 097	2 440	17 981	18 298	21,7%

Таблица 11 – Нагрузочное тестирование ручки POST /api/tasks (создание задачи)

Целевой RPS	Факт. RPS	avg, мс	p95, мс	p99, мс	Ошибки
25	25	3,1	4,8	11	0%
100	100	2,9	4,2	6,8	0%
400	400	1,3	2,4	4,1	0%
800	800	2,2	4,7	42	0%
1 600	1 600	6,0	23	157	0%
3 200	3 191	150	593	679	0%
6 400	1 733	4 563	7 352	7 727	0%

Таблица 12 – Нагрузочное тестирование ручки POST /api/routes/optimize (построение маршрута)

Целевой RPS	Факт. RPS	avg, мс	p95, мс	p99, мс	Ошибки
5	5	7,4	8,6	9,9	0%
40	40	6,2	7,8	16	0%
160	160	3,3	4,6	12	0%
640	640	3,9	6,9	78	0,02%
1 280	1 280	16	114	196	0,13%
2 560	1 225	6 113	8 653	8 708	0,63%
5 120	936	5 928	23 278	23 608	57,6%

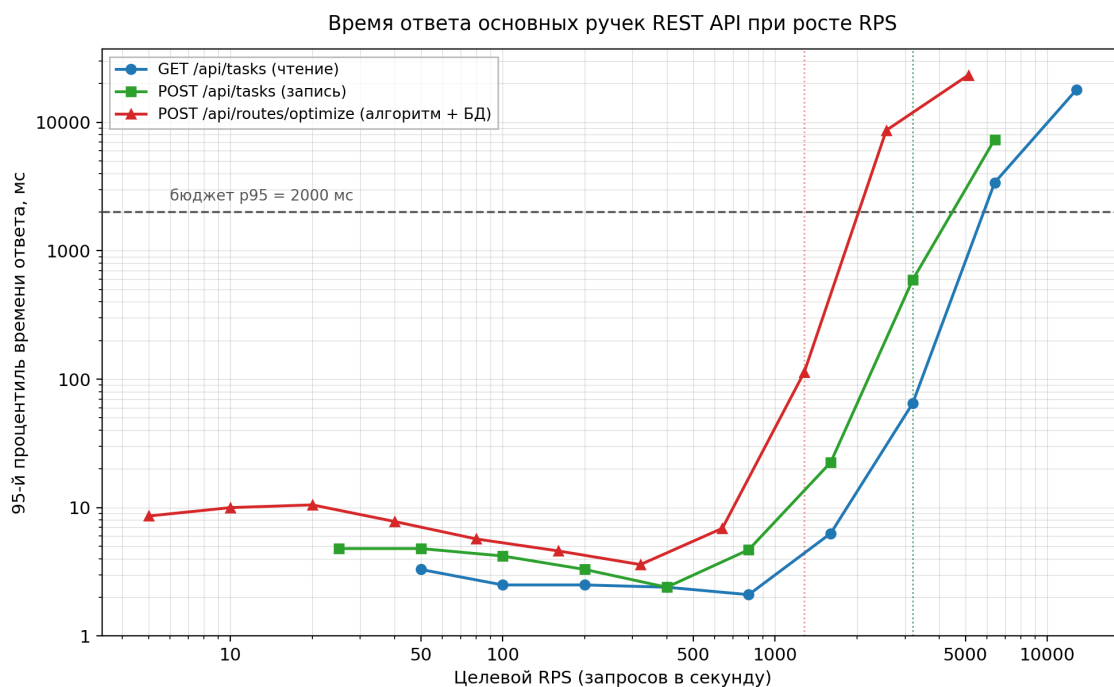


Рисунок 21 – Зависимость 95-го перцентиля
времени ответа от целевого RPS

Во всем диапазоне до точки насыщения все три ручки сохраняют низкое и почти постоянное время отклика при нулевой доле ошибок: чтение и запись удерживают p95 в пределах единиц миллисекунд вплоть до 1 600 RPS, оптимизация — единиц миллисекунд до сотен RPS. Фактический RPS точно совпадает с целевым, то есть система успевает обрабатывать запросы в темпе их поступления.

Точки насыщения по ручкам различаются и видны на рисунке 21. Чтение (GET /api/tasks) устойчиво держит до 3 200 RPS (p95 = 65 мс, ошибок нет); при 6 400 RPS система перестает успевать — фактический RPS падает до 3 860, а p95 превышает бюджет. Запись (POST /api/tasks) ведет себя схоже: до 3 200 RPS p95 = 593 мс в пределах бюджета, при 6 400 RPS наступает насыщение. Построение маршрута (POST /api/routes/optimize) — самая тяжелая операция: при прогревом кэше она устойчиво обрабатывает до 1 280 RPS (p95 = 114 мс), но при 2 560 RPS фактический RPS падает до 1 225, а p95 вырастает до 8,7 с. Более низкий предел этой ручки по сравнению с чтением и записью объясняется тем, что каждый запрос, помимо работы алгоритма

NNH-TW, перезаписывает маршрут пользователя в СУБД (удаление и вставка строк маршрута и его точек).

Полученные пределы значительно превышают нефункциональное требование о масштабируемости до 100 одновременных пользователей. Сотне пользователей с типичной активностью порядка одного запроса в секунду соответствует совокупная нагрузка около 100 RPS; даже самая дорогая ручка выдерживает примерно 13-кратный, а ручки чтения и записи — примерно 30-кратный запас по интенсивности при р95 заметно ниже порога в 2000 мс и нулевой доле ошибок. Требование масштабируемости подтверждено экспериментально с большим запасом. Следует учитывать, что генератор нагрузки и серверный стек работали на одной машине, поэтому за точкой насыщения часть деградации связана с конкуренцией за общий процессор; реальный предел развернутого на выделенном сервере приложения не ниже измеренного.

3.3 Тестирование

Тестирование алгоритма оптимизации. Стратегия тестирования следует общим принципам отбора тест-кейсов на основе классов эквивалентности, граничных значений и ветвлений алгоритма [7–8]. Наиболее критичным компонентом с точки зрения корректности является алгоритм NNH-TW. Для его проверки разработано 35 тест-кейсов, охватывающих все ветви алгоритма. Основные группы выполняемых проверок:

1. базовый обход графа: корректная обработка графов из одного, нескольких и пустого набора узлов, выбор ближайшего соседа и подсчет агрегированных показателей маршрута (расстояние, время в пути, ожидание, обслуживание);
2. учет временных окон: выбор стартового узла по наиболее раннему окну, ожидание перед открытием окна, просмотр вперед, предотвращающий

пропуск узла с жестким дедлайном, и предпочтение более срочной задачи при равном времени завершения;

3. граничные значения допустимости: прибытие при отсутствии окна, в пределах окна, до открытия, после закрытия и точно на границе окна;

4. резервный проход: выбор узла без учета окна при отсутствии допустимых кандидатов и обработка недостижимых пар точек;

5. дополнительные ограничения: фиксация начальной и конечной точек маршрута и соблюдение попарных ограничений порядка «А до В»;

6. тайминги остановок: корректность расчета времени прибытия, ожидания, начала и завершения обслуживания.

Тестирование бизнес-логики серверной части. Бизнес-логика тестируется с использованием репозиторий-заглушек, генерируемых пакетом `testify/mock` [41], что позволяет проверять варианты использования в изоляции от базы данных. Покрываются следующие компоненты:

1. `auth/story` — регистрация, вход, выход, обновление токена;
2. `task/story` — создание, редактирование, удаление задачи, проверка принадлежности задачи пользователю;
3. `route/story` — запуск оптимизации и сохранение единственного маршрута пользователя.

Тестирование клиентской части. Клиентская часть тестируется средствами `vitest` [60] — среды тестирования на базе `Vite` [19], библиотеки `@testing-library/react` [61] и `msw` (Mock Service Worker) [57] — для перехвата HTTP-запросов в тестах. Ключевые тест-кейсы:

1. Логика обновления токена: при ответе 401 запускается ровно один запрос обновления (дедупликация через общий `Promise`); после успешного обновления исходный запрос повторяется;
2. Утилита построения матрицы расстояний: формирование $n \times (n - 1)$ запросов к мультимаршрутизатору Яндекс JS API.

3.4 Внедрение и сопровождение программного продукта

Приложение является веб-приложением с клиент-серверной архитектурой; конечный пользователь взаимодействует с ним через стандартный веб-браузер без установки дополнительного программного обеспечения. Развертывание серверной части через Docker Compose и сборка клиентской части командой `npm build` рассмотрены выше. Перед переходом в продуктивную эксплуатацию обязательно меняются учетные данные СУБД, устанавливается криптографически стойкое значение `APP_JWT_SECRET` и задается корректный `APP_CORS_ORIGINS`.

3.5 Ограничения

В текущей реализации хранение географических координат задач в базе данных осуществляется в соответствии с условиями бесплатного тарифа Яндекс JS API [17], которые допускают это для учебных и некоммерческих проектов; для коммерческого использования потребуется переход на платную лицензию. Суточный лимит бесплатного тарифа составляет 1 000 запросов к геокодеру и 25 000 запросов к JavaScript API; при превышении указанных лимитов рекомендуется переход на коммерческий тариф либо интеграция с альтернативным геокодером.

3.6 Выводы

Реализованная система соответствует функциональным и нефункциональным требованиям, сформулированным в первой главе.

Архитектурные решения — кэширование матрицы расстояний, хранение паролей и токенов обновления в виде хэшей, автоматическое восстановление сессии — обеспечивают отказоустойчивость и надежность работы. Стратегия тестирования охватывает алгоритм оптимизации (35 тест-кейсов с

проверкой временных окон, просмотра вперед и попарных ограничений порядка), бизнес-логику серверной части (тесты через заглушки `testify/mock`) и клиентский API-клиент (тесты на стеке `vitest` с `MSW`).

Нагрузочное тестирование по открытой модели подтвердило выполнение требования масштабируемости с не менее чем 13-кратным запасом относительно 100 одновременных пользователей при 95-м процентиле времени ответа в пределах бюджета 2 000 мс и нулевой доле ошибок.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы разработано веб-приложение для автоматизированного планирования маршрутов с учетом временных окон, географического положения точек и длительности выполнения задач.

В результате работы решены следующие задачи:

1. Проведен анализ предметной области: выявлены ограничения существующих приложений для управления задачами и навигационных сервисов, обоснована актуальность разработки.

2. Исследованы алгоритмические подходы к решению задачи маршрутизации с временными окнами; выбрана и реализована эвристика ближайшего соседа с одношаговым просмотром вперед (NNH-TW) с временной сложностью $O(n^3)$.

3. Разработана архитектура программной системы на основе принципов чистой архитектуры с разделением на слои: доменная логика, варианты использования, HTTP-доставка и репозитории базы данных.

4. Реализована серверная часть на языке Go с использованием фреймворка Gin, предоставляющая REST API для управления задачами, маршрутами и аутентификацией пользователей. Алгоритм оптимизации поддерживает фиксацию начальной и конечной точки маршрута, а также попарные ограничения порядка посещения.

5. Разработан пользовательский интерфейс на React с отображением задач в виде карточек, интерактивной картой Яндекс JS API, выбором режима транспорта, ручным упорядочиванием методом перетаскивания, импортом задач из CSV и Excel, экспортом в CSV и XLSX и визуальным обнаружением конфликтов временных окон.

6. Реализована интеграция с сервисом «Яндекс JavaScript API и HTTP Геокодер» версии 2.1 для геокодирования адресов с автодополнением и ви-

зуализации маршрута, а также с OSRM в качестве резервного источника матрицы расстояний.

7. Проведено тестирование решения: 35 модульных тестов алгоритма оптимизации, тесты бизнес-логики и HTTP-обработчиков серверной части, тесты клиентского API-клиента.

8. Выполнена оценка производительности и масштабируемости: нагрузочное тестирование по открытой модели подтвердило выполнение требования о 100 одновременных пользователях с многократным запасом (95-й процентиль времени ответа в пределах бюджета 2 000 мс, ошибок нет).

Поставленная цель достигнута: разработанное веб-приложение объединяет управление задачами, автоматическую оптимизацию маршрута с учетом временных окон и визуализацию на карте. По сравнению с менеджерами задач (Todoist, Microsoft To Do) система дополнительно реализует геокодирование адресов и оптимизацию порядка посещения; по сравнению с навигационными сервисами (Google Maps, Яндекс Карты) — учет временных окон и длительности обслуживания; по сравнению с логистическими платформами (Routific, Яндекс Маршрутизация) — доступность для индивидуального пользователя без подписки.

Практическая значимость работы заключается в предоставлении частным пользователям, специалистам на выезде, курьерам малого бизнеса и индивидуальным предпринимателям единого инструмента планирования маршрутов с временными окнами, не требующего коммерческой подписки. Веб-форма распространения и контейнеризация на основе Docker Compose обеспечивают воспроизводимое развертывание на типовом серверном оборудовании и не предъявляют требований к рабочему месту пользователя помимо современного веб-браузера.

Перспективные направления развития:

1. поддержка командной работы с общим пулом задач и распределенной ответственностью;

2. интеграция с внешними календарями (Google Calendar, Яндекс Календарь) для двунаправленной синхронизации задач;
3. реализация фоновых уведомлений о наступлении временных окон ближайших задач;
4. внедрение метаэвристик (муравьиный алгоритм, генетический алгоритм) для повышения качества маршрутов при большом числе задач — интерфейс Optimizer в коде серверной части рассчитан на такую замену;
5. учет реального дорожного трафика при формировании матрицы расстояний.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Шкодинский С. В., Козлова А. Р. Ключевые аспекты логистики последней мили в онлайн-торговле // Прикладные экономические исследования. — 2025. — № 5. — С. 223–228. — DOI: 10.47576/2949-1908.2025.5.5.026.
2. Бронникова Е. М. Использование информационных технологий в тайм-менеджменте // Научные труды Вольного экономического общества России. — Москва: ВЭО России, 2010.
3. Бондаренко Т. В., Гарибов А. И., Федотов Е. А. Решение задачи планирования маршрутов с учетом временных окон // Инженерный вестник Дона. — Ростов-на-Дону, 2020. — № 5 (65).
4. Мартин Р. К. Чистая архитектура: искусство разработки программного обеспечения. — Санкт-Петербург: Питер, 2018. — 352 с. — (Серия «Библиотека программиста»). — ISBN 978-5-4461-0772-8.
5. Эванс Э. Предметно-ориентированное проектирование (DDD): структуризация сложных программных систем. — Москва: Вильямс, 2011. — 448 с. — ISBN 978-5-8459-1597-9.
6. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. — Санкт-Петербург: Питер, 2009. — 366 с. — ISBN 978-5-469-01136-1.
7. Куликов С. С. Тестирование программного обеспечения. Базовый курс. — 2-е изд. — Минск: Четыре четверти, 2017. — 312 с. — ISBN 978-985-7081-77-0.
8. Майерс Г., Сандлер К., Баджетт Т. Искусство тестирования программ. — 3-е изд. — Москва: Диалектика, 2018. — 272 с. — ISBN 978-5-8459-1796-6.
9. Alessandretti L., Szell M. Urban Mobility // Compendium of Urban Complexity. — Cham: Springer, 2025. — P. 57–76. — DOI: 10.1007/978-3-031-82666-5_4.

10. Левкин Г. Г., Куликова О. М. Аспекты управления логистикой на этапе последней мили // Вестник Евразийской науки. — 2021. — Т. 13.
11. The Go Programming Language [Электронный ресурс]. — URL: <https://go.dev/> (дата обращения: 15.03.2026).
12. Saioc G.-V., Shirchenko D., Chabbi M. Unveiling and Vanquishing Goroutine Leaks in Enterprise Microservices: A Dynamic Analysis Approach // Proceedings of the IEEE/ACM 46th International Conference on Software Engineering: Companion (ICSE-Companion 2024). — New York: ACM, 2024. — P. 314–325. — DOI: 10.1145/3639478.3640033.
13. PostgreSQL 16 Documentation [Электронный ресурс]. — The PostgreSQL Global Development Group, 2026. — URL: <https://www.postgresql.org/docs/16/index.html> (дата обращения: 15.03.2026).
14. Черный Б. Программирование на TypeScript. — Санкт-Петербург: Питер, 2020. — 304 с. — (Серия «Бестселлеры O'Reilly»). — ISBN 978-5-4461-1601-0.
15. Бэнкс А., Порселло Е. React и Redux: функциональная веб-разработка. — Санкт-Петербург: Питер, 2018. — 336 с. — ISBN 978-5-4461-0668-4.
16. Bogner J., Merkel M. To Type or Not to Type? A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub // Proceedings of the 19th International Conference on Mining Software Repositories (MSR 2022). — New York: ACM, 2022. — P. 658–669. — DOI: 10.1145/3524842.3528454.
17. Яндекс JavaScript API и HTTP Геокодер [Электронный ресурс]. — URL: <https://yandex.ru/dev/jsapi-v2-1/doc/ru/> (дата обращения: 12.04.2026).
18. Gin Web Framework [Электронный ресурс]. — URL: <https://gin-gonic.com/> (дата обращения: 11.04.2026).
19. Vite: Next Generation Frontend Tooling [Электронный ресурс]. — URL: <https://vite.dev/> (дата обращения: 11.04.2026).

20. Моуэт А. Использование Docker. — Москва: ДМК Пресс, 2017. — 354 с. — ISBN 978-5-97060-426-7.
21. Held M., Karp R. M. A Dynamic Programming Approach to Sequencing Problems // Journal of the Society for Industrial and Applied Mathematics. — 1962. — Vol. 10, No. 1. — P. 196–210. — DOI: 10.1137/0110015.
22. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — Москва: Вильямс, 2013. — 1328 с. — ISBN 978-5-8459-1794-2.
23. Ахо А., Хопкрофт Дж., Ульман Дж. Структуры данных и алгоритмы. — Москва: Вильямс, 2010. — 400 с. — ISBN 978-5-8459-1610-5.
24. Скиена С. С. Алгоритмы. Руководство по разработке. — 3-е изд. — Санкт-Петербург: БХВ-Петербург, 2021. — 720 с. — ISBN 978-5-9775-6748-2.
25. Седжвик Р., Уэйн К. Алгоритмы на Java. — 4-е изд. — Москва: Вильямс, 2013. — 848 с. — ISBN 978-5-8459-1781-2.
26. Гвоздев Л. Р., Медведева Т. А. Решение задачи маршрутизации транспортных средств с временными окнами с помощью алгоритма муравьиных колоний // Молодой исследователь Дона. — Ростов-на-Дону: ДГТУ, 2022. — № 3 (36). — С. 58–61.
27. Necula R., Breaban M., Raschip M. Tackling Dynamic Vehicle Routing Problem with Time Windows by means of Ant Colony System // Proceedings of the 2017 IEEE Congress on Evolutionary Computation (CEC 2017). — Donostia: IEEE, 2017. — P. 2480–2487. — DOI: 10.1109/CEC.2017.7969606.
28. Гладков Л. А., Курейчик В. В., Курейчик В. М. Генетические алгоритмы: учебник / под ред. В. М. Курейчика. — 2-е изд., испр. и доп. — Москва: ФИЗМАТЛИТ, 2010. — 368 с. — ISBN 978-5-9221-0510-1.
29. Todoist: Organize your work and life, finally [Электронный ресурс]. — URL: <https://todoist.com/pricing> (дата обращения: 12.04.2026).

30. Microsoft To Do: Tasks, To-Do lists & Reminders [Электронный ресурс]. — URL: <https://to-do.microsoft.com/> (дата обращения: 12.04.2026).
31. Singularity [Электронный ресурс]. — URL: <https://singularity-app.ru/> (дата обращения: 12.04.2026).
32. Google Maps [Электронный ресурс]. — URL: <https://maps.google.com/> (дата обращения: 12.04.2026).
33. API Яндекс Карты: документация для разработчиков [Электронный ресурс]. — URL: <https://yandex.ru/dev/maps/> (дата обращения: 24.04.2026).
34. 2ГИС: документация для разработчиков [Электронный ресурс]. — URL: <https://docs.2gis.com/ru/> (дата обращения: 24.04.2026).
35. Google Maps Help. Getting directions and showing routes [Электронный ресурс]. — URL: <https://support.google.com/maps/answer/144339> (дата обращения: 12.04.2026).
36. Routific: Intelligent Route Optimization Software [Электронный ресурс]. — URL: <https://routific.com/pricing> (дата обращения: 12.04.2026).
37. Relog — сервис оптимизации маршрутов [Электронный ресурс]. — URL: <https://relog.ru/> (дата обращения: 20.03.2026).
38. Яндекс Маршрутизация — сервис автоматического построения маршрутов [Электронный ресурс]. — URL: <https://routing.yandex.ru/> (дата обращения: 20.03.2026).
39. Spoke — Route Optimization Software [Электронный ресурс]. — URL: <https://www.spokeapp.io/> (дата обращения: 20.03.2026).
40. pnpm: Fast, disk space efficient package manager [Электронный ресурс]. — URL: <https://pnpm.io/> (дата обращения: 07.05.2026).
41. stretchr/testify: A toolkit with common assertions and mocks that plays nicely with the standard library [Электронный ресурс]. — URL: <https://github.com/stretchr/testify> (дата обращения: 07.05.2026).
42. HashiCorp. golang-lru: Golang LRU cache [Электронный ресурс]. — URL: <https://github.com/hashicorp/golang-lru> (дата обращения: 12.04.2026).

43. Кудухов А. Н. К вопросу о применении алгоритмов замещения LRU в подсистемах кэширования информационных систем промышленных предприятий // Перспективы развития информационных технологий: сборник материалов XXI Международной научно-практической конференции. — Новосибирск: ЦРНС, 2014.
44. Гэри М., Джонсон Д. Вычислительные машины и труднорешаемые задачи. — Москва: Мир, 1982. — 416 с.
45. Tailwind CSS: Responsive design [Электронный ресурс]. — URL: <https://tailwindcss.com/docs/responsive-design> (дата обращения: 07.05.2026).
46. shadcn/ui: A set of beautifully-designed, accessible components and a code distribution platform [Электронный ресурс]. — URL: <https://ui.shadcn.com/docs> (дата обращения: 07.05.2026).
47. Radix Primitives: Unstyled, accessible, open source React primitives for high-quality web apps and design systems [Электронный ресурс]. — URL: <https://www.radix-ui.com/primitives> (дата обращения: 07.05.2026).
48. The Linux Kernel Archives [Электронный ресурс] / The Linux Kernel Organization. — URL: <https://www.kernel.org/> (дата обращения: 07.05.2026).
49. macOS [Электронный ресурс] / Apple Inc. — URL: <https://www.apple.com/macos/> (дата обращения: 07.05.2026).
50. Windows 11: Features and Benefits [Электронный ресурс] / Microsoft Corporation. — URL: <https://www.microsoft.com/en-us/windows/windows-11> (дата обращения: 07.05.2026).
51. Node.js Documentation [Электронный ресурс] / OpenJS Foundation. — URL: <https://nodejs.org/docs/> (дата обращения: 07.05.2026).
52. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT): RFC 7519. — Internet Engineering Task Force, 2015. — 30 p.
53. Hardt D. The OAuth 2.0 Authorization Framework: RFC 6749. — Internet Engineering Task Force, 2012. — 76 p.

54. Secure Hash Standard (SHS): Federal Information Processing Standards Publication FIPS PUB 180-4 / National Institute of Standards and Technology. — Gaithersburg: U.S. Department of Commerce, 2015. — 36 p. — DOI: 10.6028/NIST.FIPS.180-4.

55. Девицына С. Н., Пилькевич П. В., Удод Е. В. Способы улучшения защищенности сервисов, использующих JWT-токены // Экономика. Информатика. — 2023. — Т. 50, № 1. — С. 144–151. — DOI: 10.52575/2687-0932-2023-50-1-144-151.

56. Provos N., Mazieres D. A Future-Adaptable Password Scheme // Proceedings of the FREENIX Track: 1999 USENIX Annual Technical Conference. — Monterey: USENIX Association, 1999. — P. 81–91.

57. Mock Service Worker: API mocking library for browser and Node.js [Электронный ресурс]. — URL: <https://mswjs.io/docs/> (дата обращения: 07.05.2026).

58. golang-migrate/migrate: Database migrations written in Go [Электронный ресурс]. — URL: <https://github.com/golang-migrate/migrate> (дата обращения: 07.05.2026).

59. Grafana Labs. k6: Modern load testing for developers [Электронный ресурс]. — URL: <https://k6.io/> (дата обращения: 12.04.2026).

60. Vitest: Next Generation testing framework powered by Vite [Электронный ресурс]. — URL: <https://vitest.dev/guide/> (дата обращения: 07.05.2026).

61. React Testing Library: Simple and complete testing utilities that encourage good testing practices [Электронный ресурс]. — URL: <https://testing-library.com/docs/react-testing-library/intro/> (дата обращения: 07.05.2026).

ПРИЛОЖЕНИЕ А

Листинг SQL-миграции и конфигурации (справочное)

Схема базы данных (SQL-миграция)

Листинг А.1 – Миграция 0001_init.up.sql – инициализация схемы базы данных

```
1 CREATE EXTENSION IF NOT EXISTS "uuid-osspl";

2 -- users
3 CREATE TABLE users (
4     id                UUID                PRIMARY KEY DEFAULT
      → uuid_generate_v4(),
5     email             VARCHAR(255) NOT NULL UNIQUE,
6     password_hash     VARCHAR(255) NOT NULL,
7     created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),
8     updated_at        TIMESTAMPTZ NOT NULL DEFAULT now()
9 );

10 -- refresh_tokens (ротация токенов)
11 CREATE TABLE refresh_tokens (
12     id                UUID                PRIMARY KEY DEFAULT
      → uuid_generate_v4(),
13     user_id           UUID                NOT NULL REFERENCES users(id)
      → ON DELETE CASCADE,
14     token_hash        VARCHAR(255) NOT NULL UNIQUE,
15     expires_at        TIMESTAMPTZ NOT NULL,
16     revoked_at        TIMESTAMPTZ NULL,
17     created_at        TIMESTAMPTZ NOT NULL DEFAULT now()
18 );
19 CREATE INDEX refresh_tokens_user_id_idx ON
      → refresh_tokens(user_id);

20 -- tasks
21 CREATE TABLE tasks (
22     id                UUID                PRIMARY KEY DEFAULT
      → uuid_generate_v4(),
23     user_id           UUID                NOT NULL REFERENCES
      → users(id) ON DELETE CASCADE,
24     title             VARCHAR(200) NOT NULL,
25     address_text       VARCHAR(400) NOT NULL,
26     latitude          NUMERIC(9,6) NULL,
27     longitude          NUMERIC(9,6) NULL,
28     duration_min      INT                NULL,
```



```

29     window_start_date DATE          NULL,
30     window_start_time TIME         NULL,
31     window_end_date   DATE          NULL,
32     window_end_time   TIME         NULL,
33     sort_index         INT           NOT NULL DEFAULT 0,
34     is_completed       BOOLEAN       NOT NULL DEFAULT
→ false,
35     created_at         TIMESTAMPTZ   NOT NULL DEFAULT
→ now(),
36     updated_at         TIMESTAMPTZ   NOT NULL DEFAULT
→ now()
37 );
38 CREATE INDEX tasks_user_id_idx ON tasks(user_id);
39 CREATE INDEX tasks_user_sort_idx ON tasks(user_id,
→ sort_index);

40 -- routes (один маршрут на пользователя; агрегированные
→ метрики хранятся прямо в строке)
41 CREATE TABLE routes (
42     id                UUID           PRIMARY KEY DEFAULT
→ uuid_generate_v4(),
43     user_id           UUID           NOT NULL UNIQUE
→ REFERENCES users(id) ON DELETE CASCADE,
44     algorithm         VARCHAR(50)    NOT NULL,
45     total_distance_m  INT            NOT NULL,
46     total_travel_sec  INT            NOT NULL,
47     total_service_sec INT            NOT NULL,
48     total_wait_sec    INT            NOT NULL,
49     computed_at       TIMESTAMPTZ    NOT NULL DEFAULT now()
50 );

51 -- route_stops (упорядоченные остановки маршрута)
52 CREATE TABLE route_stops (
53     id                UUID           PRIMARY KEY DEFAULT
→ uuid_generate_v4(),
54     route_id          UUID           NOT NULL REFERENCES
→ routes(id) ON DELETE CASCADE,
55     task_id           UUID           NOT NULL REFERENCES
→ tasks(id) ON DELETE CASCADE,
56     position          INT            NOT NULL,
57     travel_from_prev_sec INT         NULL,
58     arrive_time       TIMESTAMPTZ    NULL,
59     service_start_time TIMESTAMPTZ    NULL,
60     service_end_time  TIMESTAMPTZ    NULL,
61     wait_sec          INT            NULL
62 );
63 CREATE INDEX          route_stops_route_id_idx ON
→ route_stops(route_id);
64 CREATE UNIQUE INDEX route_stops_route_position_uq ON
→ route_stops(route_id, position);

```

Конфигурация контейнеризации

Листинг А.2 – Конфигурация docker-compose.yml серверной части проекта

```
1 services:
2   db:
3     image: postgres:16
4     environment:
5       POSTGRES_DB: ${POSTGRES_DB}
6       POSTGRES_USER: ${POSTGRES_USER}
7       POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
8     ports:
9       - "5432:5432"
10    volumes:
11      - db_data:/var/lib/postgresql/data
12    healthcheck:
13      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}
→ -d ${POSTGRES_DB}"]
14      interval: 5s
15      timeout: 3s
16      retries: 20
17
18    migrate:
19      image: migrate/migrate:v4.17.1
20      depends_on:
21        db:
22          condition: service_healthy
23      volumes:
24        - ./migrations:/migrations
25      entrypoint:
26        - "migrate"
27        - "-path"
28        - "/migrations"
29        - "-database"
30        - "postgres://${POSTGRES_USER}:${POSTGRES_PASSWORD}]
→ @db:5432/${POSTGRES_DB}?sslmode=disable"
31        - "up"
32
33    api:
34      build: .
35      depends_on:
36        db:
37          condition: service_healthy
38        migrate:
39          condition: service_completed_successfully
40      environment:
41        APP_HTTP_ADDR: "${APP_HTTP_ADDR}"
42        APP_DB_DSN: "postgres://${POSTGRES_USER}:${POSTGRES_
→ _PASSWORD}@db:5432/${POSTGRES_DB}?sslmode=disable"
```

```
41     APP_JWT_SECRET: "${APP_JWT_SECRET}"
42     APP_ACCESS_TTL_MIN: "${APP_ACCESS_TTL_MIN}"
43     APP_REFRESH_TTL_DAYS: "${APP_REFRESH_TTL_DAYS}"
44     APP_CORS_ORIGINS: "${APP_CORS_ORIGINS}"
45     APP_OSRM_BASE_URL: "${APP_OSRM_BASE_URL}"
46     ports:
47         - "8080:8080"

48 volumes:
49     db_data:
```

ПРИЛОЖЕНИЕ Б

Листинги ключевых фрагментов кода (справочное)

Листинг Б.1 – Алгоритм ближайшего соседа с временными окнами и просмотром вперед (nearest_neighbor.go)

```
1 package optimizer

2 import (
3     "context"
4     "math"
5 )

6 // NearestNeighborTW – эвристика ближайшего соседа с
7   ↳ учетом временных окон
8 // (NNH-TW). Все временные величины хранятся в секундах
9   ↳ Unix. Сложность –  $O(n^3)$ .
10 type NearestNeighborTW struct{}

11 func NewNearestNeighborTW() *NearestNeighborTW { return
12   ↳ &NearestNeighborTW{} }
13 func (a *NearestNeighborTW) Name() string { return
14   ↳ "nearest-neighbor-tw" }

15 // traversal инкапсулирует состояние обхода: текущая
16   ↳ позиция, тайминги,
17 // агрегированные показатели; appendNode инкрементально
18   ↳ его обновляет.
19 type traversal struct {
20     g
21     ↳ *Graph
22     cur
23     ↳ int
24     currentTime
25     ↳ int64
26     visited
27     ↳ []bool
28     order
29     ↳ []int
30     timings
31     ↳ []StopTiming
32     totalDistM, totalTravelSec, totalServiceSec,
33     ↳ totalWaitSec int
34 }

35 func (t *traversal) appendNode(next int) {
36     e := t.g.Edges[t.cur][next]
```

```

24     node := t.g.Nodes[next]
25     arrivalSec := t.currentTime + int64(e.DurationSec)
26     var waitSec int64
27     if node.WindowStart >= 0 && arrivalSec <
→ node.WindowStart {
28         waitSec = node.WindowStart - arrivalSec
29     }
30     serviceStart := arrivalSec + waitSec
31     serviceEnd := serviceStart +
→ int64(node.DurationMin)*60

32     t.totalDistM += e.DistanceM
33     t.totalTravelSec += e.DurationSec
34     t.totalServiceSec += node.DurationMin * 60
35     t.totalWaitSec += int(waitSec)

36     t.timings = append(t.timings, StopTiming{
37         NodeIdx: next, ArrivalSec: arrivalSec, WaitSec:
→ int(waitSec),
38         ServiceStartSec: serviceStart, ServiceEndSec:
→ serviceEnd,
39         TravelFromPrevSec: e.DurationSec,
40     })
41     t.visited[next] = true
42     t.order = append(t.order, next)
43     t.cur = next
44     t.currentTime = serviceEnd
45 }

46 func (a *NearestNeighborTW) Optimize(
47     _ context.Context, g *Graph, startTimeUnix int64, c
→ Constraints,
48 ) (Result, error) {
49     n := len(g.Nodes)
50     if n == 0 {
51         return Result{}, nil
52     }
53     prereqs := buildPrereqs(n, c.PrecedencePairs)
54     endIdx := -1
55     if c.EndNodeIdx != nil {
56         endIdx = *c.EndNodeIdx
57     }

58     cur := 0
59     if c.StartNodeIdx != nil {
60         cur = *c.StartNodeIdx
61     } else {
62         cur = startNode(g, prereqs, endIdx)
63     }
64     startServiceEnd := startTimeUnix +
→ int64(g.Nodes[cur].DurationMin)*60

```

```

65     t := &traversal{
66         g: g, cur: cur, currentTime: startServiceEnd,
67         visited: make([]bool, n), order: []int{cur},
68         timings: []StopTiming{{
69             NodeIdx: cur, ArrivalSec: startTimeUnix,
70             ServiceStartSec: startTimeUnix,
71             ServiceEndSec: startServiceEnd,
72             totalServiceSec: g.Nodes[cur].DurationMin * 60,
73         }}
74     t.visited[cur] = true

75     for len(t.order) < n {
76         // Если остался только закрепленный конечный узел
77         - ставим его и выходим.
78         if endIdx >= 0 && !t.visited[endIdx] &&
79         onlyEndUnvisited(t.visited, endIdx) {
80             t.appendNode(endIdx)
81             break
82         }
83         next := pickNextFeasible(t, prereqs, endIdx)
84         if next == -1 {
85             next = pickNextFallback(t, prereqs, endIdx)
86         }
87         if next == -1 {
88             // Циклический граф предшествования -
89             конечный узел добавим и завершим.
90             if endIdx >= 0 && !t.visited[endIdx] {
91                 t.appendNode(endIdx)
92             }
93             break
94         }
95         t.appendNode(next)
96     }
97     return Result{
98         Order: t.order, Timings: t.timings,
99         TotalDistanceM: t.totalDistM, TotalTravelSec:
100         t.totalTravelSec,
101         TotalServiceSec: t.totalServiceSec, TotalWaitSec:
102         t.totalWaitSec,
103     }, nil
104 }

100 // pickNextFeasible - проход 1: допустимый узел с
101 // минимальным временем
102 // завершения + look-ahead. Безопасные кандидаты
103 // предпочитают.
104 func pickNextFeasible(t *traversal, prereqs [][]int,
105     endIdx int) int {
106     g := t.g
107     next := -1

```

```

105     var bestComp int64 = math.MaxInt64
106     var bestDeadline int64 = math.MaxInt64
107     bestSafe := false
108     for i := range g.Nodes {
109         if t.visited[i] || i == endIdx || !prereqsMet(i,
→ t.visited, prereqs) {
110             continue
111         }
112         arrival := t.currentTime +
→ int64(g.Edges[t.cur][i].DurationSec)
113         if !feasible(g.Nodes[i], arrival) {
114             continue
115         }
116         comp := nodeCompletionTime(g.Nodes[i], arrival)
117         deadline := g.Nodes[i].WindowEnd
118         if deadline < 0 {
119             deadline = math.MaxInt64
120         }
121         safe := !causesWindowMiss(i, comp, g, t.visited,
→ endIdx)
122         better := false
123         switch {
124             case safe && !bestSafe:
125                 better = true
126             case safe == bestSafe:
127                 better = comp < bestComp ||
128                     (comp == bestComp && deadline <
→ bestDeadline)
129         }
130         if better {
131             bestComp = comp; bestDeadline = deadline
132             bestSafe = safe; next = i
133         }
134     }
135     return next
136 }

137 // pickNextFallback — проход 2: минимальный
→ completionTime, окно игнорируется,
138 // предшествование сохраняется.
139 func pickNextFallback(t *traversal, prereqs [][]int,
→ endIdx int) int {
140     g := t.g
141     next := -1
142     var bestComp int64 = math.MaxInt64
143     for i := range g.Nodes {
144         if t.visited[i] || i == endIdx || !prereqsMet(i,
→ t.visited, prereqs) {
145             continue
146         }

```

```

147         arrival := t.currentTime +
    ↪   int64(g.Edges[t.cur][i].DurationSec)
148         comp := nodeCompletionTime(g.Nodes[i], arrival)
149         if comp < bestComp {
150             bestComp = comp; next = i
151         }
152     }
153     return next
154 }

155 func feasible(node Node, arrivalSec int64) bool {
156     if node.WindowEnd < 0 {
157         return true
158     }
159     return arrivalSec <=
    ↪   node.WindowEnd-int64(node.DurationMin)*60
160 }

161 // causesWindowMiss — look-ahead на один шаг: true, если
    ↪   после cand
162 // хотя бы один узел с окном становится недостижимым.
163 func causesWindowMiss(cand int, afterSec int64,
164     g *Graph, visited []bool, endIdx int) bool {
165     for j := 0; j < len(g.Nodes); j++ {
166         if visited[j] || j == cand || j == endIdx {
167             continue
168         }
169         if g.Nodes[j].WindowEnd < 0 {
170             continue
171         }
172         arr := afterSec +
    ↪   int64(g.Edges[cand][j].DurationSec)
173         if !feasible(g.Nodes[j], arr) {
174             return true
175         }
176     }
177     return false
178 }

```

Листинг Б.2 – Ключевые функции API-клиента: оптимизация маршрута и обновление токена (client.ts)

```

1 let refreshSessionPromise: Promise<AuthSession> | null =
    ↪   null;

2 export interface PrecedenceConstraint {
3     beforeTaskId: string;
4     afterTaskId: string;

```



```

5 }

6 // optimizeRoute передает идентификаторы задач, матрицу
  → расстояний
7 // и дополнительные ограничения на бэкенд.
8 export async function optimizeRoute(
9   taskIds: string[],
10  options: AuthorizedRequestOptions,
11  startTimeUnix = 0,
12  distanceMatrix?: DistanceCell[][],
13  startTaskId?: string,
14  endTaskId?: string,
15  precedenceConstraints?: PrecedenceConstraint[],
16 ): Promise<SavedRouteDetails> {
17   const route = await requestWithAuth<ApiRouteFull>(
18     '/routes/optimize',
19     {
20       method: 'POST',
21       body: JSON.stringify({
22         taskIds, startTimeUnix, distanceMatrix,
23         startTaskId: startTaskId ?? undefined,
24         endTaskId: endTaskId ?? undefined,
25         precedenceConstraints:
26           → precedenceConstraints?.length
27             ? precedenceConstraints : undefined,
28       }),
29       options,
30     });
31   return mapRouteDetails(route);
32 }

33 // requestWithAuth выполняет HTTP-запрос с токеном
  → доступа.
34 // При получении 401 однократно обновляет токен и
  → повторяет запрос.
35 async function requestWithAuth<T>(
36   path: string, init: RequestInit,
37   options: AuthorizedRequestOptions,
38 ): Promise<T> {
39   try {
40     return await request<T>(path, init,
41       → options.session.accessToken);
42   } catch (error) {
43     if (!(error instanceof ApiError) || error.status !==
44       → 401) throw error;
45   }
46   try {
47     const nextSession = await refreshSession(options);
48     return request<T>(path, init,
49       → nextSession.accessToken);

```

```

47   } catch (error) {
48     await options.onUnauthorized();
49     throw error;
50   }
51 }

52 // refreshSession — дедупликация через единственный
   → промис.
53 async function refreshSession(options:
   → AuthorizedRequestOptions) {
54   if (!refreshSessionPromise) {
55     refreshSessionPromise =
   → request<TokenPairResponse>('/auth/refresh', {
56       method: 'POST',
57       body: JSON.stringify({
58         refreshToken: options.session.refreshToken,
59       }),
60     })
61     .then((pair) => {
62       const nextSession = buildSession(pair,
63         getEmailFromToken(pair.accessToken) ??
   → options.session.email);
64       options.onSessionChange(nextSession);
65       return nextSession;
66     })
67     .finally(() => { refreshSessionPromise = null; });
68   }
69   return refreshSessionPromise;
70 }

```

Листинг Б.3 – Построение матрицы расстояний через Яндекс Маршруты JS API (routeOptimizer.ts)

```

1 import { Task } from '../types/task';
2 import { loadYandexMaps } from '../yandexMaps';
3 import { roundUpSecToMinute } from '../time';

4 export interface DistanceCell {
5   distanceM: number;
6   durationSec: number;
7 }

8 /**
9  * Строит матрицу расстояний n×n для списка задач через
   → Яндекс Маршруты.
10 * Используется тот же движок, что и для визуализации
   → маршрута на карте,
11 * поэтому данные оптимизации согласованы с отображением.

```

```

12  *
13  * matrix[i][j] = стоимость проезда от tasks[i] до
    → tasks[j].
14  * Диагональные элементы (i === j) равны нулю.
15  */
16  export async function buildYandexDistanceMatrix(
17    tasks: Task[],
18    routingMode: 'auto' | 'pedestrian' | 'masstransit' =
    → 'auto',
19  ): Promise<DistanceCell[][]> {
20    const n = tasks.length;
21    if (n === 0) return [];

22    await loadYandexMaps();

23    // masstransit через multiRouter не возвращает надежные
    → данные по времени —
24    // используем 'auto' как приближение.
25    const mode = routingMode === 'masstransit' ? 'auto' :
    → routingMode;

26    const fallbackSec = 99_999;
27    const matrix: DistanceCell[][] = Array.from({ length:
    → n }, (_, i) =>
28      Array.from({ length: n }, (_, j) =>
29        i === j
30          ? { distanceM: 0, durationSec: 0 }
31          : { distanceM: 0, durationSec: fallbackSec },
32      ),
33    );

34    // Запрашиваем все пары параллельно: n*(n-1) запросов.
35    const pairs: [number, number][] = [];
36    for (let i = 0; i < n; i++)
37      for (let j = 0; j < n; j++)
38        if (i !== j) pairs.push([i, j]);

39    await Promise.all(
40      pairs.map(async ([i, j]) => {
41        const from = tasks[i];
42        const to = tasks[j];
43        if (from.latitude == null || to.latitude == null)
    → return;

44        try {
45          const { distanceM, durationSec } = await new
    → Promise<{
46            distanceM: number;
47            durationSec: number;
48          }>((resolve, reject) => {

```

```

49         const mr = new (window.ymaps as
→ any).multiRouter.MultiRoute(
50             {
51                 referencePoints: [
52                     [from.latitude!, from.longitude!],
53                     [to.latitude!, to.longitude!],
54                 ],
55                 params: { routingMode: mode },
56             },
57             {},
58         );

59         mr.model.events.once('requestsuccess', () => {
60             try {
61                 const activeRoute: any =
→ mr.getActiveRoute();
62                 if (activeRoute) {
63                     const distObj: any =
→ activeRoute.properties.get('distance');
64                     const durObj: any =
→ activeRoute.properties.get('duration');
65                     resolve({
66                         distanceM: Math.round(
67                             typeof distObj?.value === 'number' ?
→ distObj.value : 0,
68                         ),
69                         durationSec: roundUpSecToMinute(
70                             typeof durObj?.value === 'number'
71                             ? durObj.value
72                             : fallbackSec,
73                         ),
74                     });
75                     return;
76                 }

77                 reject(new Error('no route data'));
78             } catch (e) {
79                 reject(e);
80             }
81         });

82         mr.model.events.once('requestfail', () => {
83             reject(new Error('routing failed'));
84         });
85     });

86     matrix[i][j] = { distanceM, durationSec };
87     } catch {
88         // Оставляем fallback: алгоритм избегает
→ недостижимых пар.
89     }

```

```
90     }),  
91 );  
  
92 return matrix;  
93 }
```

ПРИЛОЖЕНИЕ В

Техническое задание на разработку (обязательное)

В.1 Общие сведения

Полное наименование системы: веб-приложение для автоматизированного планирования маршрутов с учетом временных окон. Условное обозначение – «Планировщик маршрутов». Организация-разработчик – Московский политехнический университет.

Основание для разработки – задание на выпускную квалификационную работу. Плановые сроки: февраль – май 2026 года. Прямое финансирование не предусмотрено.

Результаты передаются заказчику в виде пояснительной записки, исходного кода (репозиторий Git) и собранного дистрибутива (Docker-образ серверной части, статическая сборка клиентской части).

В.2 Назначение и цели создания системы

Назначение. Автоматизированное построение оптимальной последовательности выполнения задач с учетом географического положения точек, временных окон обслуживания и длительности выполнения каждой задачи.

Цели:

1. сокращение времени ручного планирования маршрута с временными окнами;
2. уменьшение числа нарушений временных окон за счет автоматизированной проверки допустимости;
3. доступность инструментов оптимизации маршрута для частных пользователей и малого бизнеса без коммерческой подписки.

В.3 Характеристика объекта автоматизации

Объект автоматизации – процесс планирования последовательности выполнения задач пользователя в городской среде. В отсутствие специализированного программного обеспечения планирование выполняется вручную; при увеличении числа задач трудоемкость возрастает нелинейно, а вероятность нарушения временных ограничений увеличивается.

Целевая аудитория: частные пользователи, специалисты на выезде, курьеры малого бизнеса, индивидуальные предприниматели, диспетчеры.

В.4 Требования к системе

В.4.1 Требования к системе в целом

Система реализуется в виде клиент-серверного веб-приложения и состоит из серверной части (хранение данных, бизнес-логика, алгоритмы оптимизации) и клиентской части (взаимодействие с пользователем через веб-браузер).

Численность пользователей – до 100 одновременных. Режим функционирования – непрерывный с допустимыми технологическими перерывами. Время отклика интерфейса не превышает 2 секунд при нагрузке до 5 запросов в секунду.

Надежность – сохранение работоспособности при кратковременной недоступности внешнего картографического сервиса за счет кэширования. Эргономика – адаптивный интерфейс для устройств с диагональю экрана от 4 до 32 дюймов. Безопасность – хранение паролей в виде хэш-значений bcrypt, токенов обновления – SHA-256; передача данных по HTTPS.

В.4.2 Требования к функциям

Система должна реализовать:

1. регистрацию и аутентификацию пользователей по электронной почте и паролю;
2. создание, редактирование, удаление и клонирование задач с указанием адреса, длительности и временного окна;
3. импорт и экспорт перечня задач в форматах CSV и XLSX;
4. геокодирование адресов через внешний сервис картографии;
5. построение матрицы расстояний между адресами задач;
6. автоматизированное вычисление оптимальной последовательности с учетом временных окон;
7. дополнительные ограничения: фиксация начальной и конечной точки маршрута, попарные ограничения порядка;
8. расчет времени прибытия, ожидания и завершения обслуживания для каждой остановки;
9. визуализацию маршрута на карте, обнаружение конфликтов временных окон;
10. выбор способа перемещения (автомобиль, пешеход, общественный транспорт).

В.4.3 Требования к видам обеспечения

Информационное обеспечение – СУБД PostgreSQL 16; структура управляется механизмом миграций.

Программное обеспечение серверной части – Docker Engine 24.0 и выше, Docker Compose 2.0 и выше (Linux, macOS или Windows).

Программное обеспечение клиентской части – веб-браузер с поддержкой JavaScript ES2020 (Chrome 90+, Firefox 88+, Safari 14+, Edge 90+, Yandex Browser 25+).

Техническое обеспечение серверной части – процессор не ниже 1 ГГц, ОЗУ не менее 512 МБ, дисковое пространство не менее 1 ГБ, соединение с сетью Интернет.

Лингвистическое обеспечение – язык пользовательского интерфейса – русский.

В.5 Состав и содержание работ по созданию системы

1. анализ предметной области, формирование требований;
2. проектирование архитектуры и модели данных;
3. выбор алгоритма оптимизации маршрута;
4. реализация серверной части (REST API, бизнес-логика, репозитории);
5. реализация клиентской части (интерфейс, интеграция с картографией);
6. модульное и нагрузочное тестирование;
7. подготовка эксплуатационной документации.

В.6 Порядок контроля и приемки

Приемка осуществляется в форме защиты выпускной квалификационной работы перед государственной экзаменационной комиссией: демонстрация работы системы на типовом сценарии, проверка выполнения функциональных и нефункциональных требований, проверка отчета о нагрузочном тестировании.

В.7 Требования к составу и содержанию работ по подготовке объекта автоматизации

Для ввода системы в действие необходимо:

1. обеспечить сервер с Docker Engine 24.0+ и Docker Compose 2.0+;
2. получить ключ доступа к Яндекс JavaScript API;
3. установить секретный ключ для подписи JWT-токенов (не менее 32 символов);
4. сконфигурировать список допустимых источников CORS;
5. обеспечить резервное копирование данных PostgreSQL средствами pg_dump.

В.8 Требования к документированию

1. пояснительная записка к выпускной квалификационной работе;
2. исходный код с комментариями godoc (сервер) и TSDoc (клиент);
3. файл README.md с инструкцией по развертыванию.

В.9 Источники разработки

1. ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы;
2. ГОСТ 7.32-2017. Отчет о научно-исследовательской работе;
3. ГОСТ 19.701-90. Схемы алгоритмов, программ, данных и систем;
4. публикации по теории графов и комбинаторной оптимизации (см. список использованных источников);
5. официальная документация платформ Go, PostgreSQL, React, Vite, Яндекс JS API.